

**T.C.  
SAKARYA ÜNİVERSİTESİ  
BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ**

**BSM 401 BİLGİSAYAR MÜHENDİSLİĞİ TASARIMI**

**SAĞLIK SEKTÖRÜNDE FEDERATİF ÖĞRENME TABANLI  
YAPAY ZEKA**

**B221210100 – Muhammed Yusuf ÖNGEL  
B221210065 – İsmail Alper KARADENİZ**

**Bölüm : BİLGİSAYAR MÜHENDİSLİĞİ**  
**Danışman : Dr. Öğr. Üyesi İsmail ÖZTEL**

**2016-2017 Güz Dönemi**

# ÖNSÖZ

Günümüzde sağlık sektörü, hastalıkların teşhisinde yapay zekâ teknolojilerinin kritik önemini kavramış ve kurumlar arası ilişkileri veri gizliliği esasına bağlı olarak yeniden yapılandırmaya başlamıştır. Özellikle Federatif Öğrenme mimarisiyle geliştirilen bu teknolojik iş birliğinin; hasta mahremiyetinin korunması, veri güvenliğinin sağlanması, yasal kısıtlamaların aşılması ve teşhis doğruluk oranlarının artırılması gibi konularda son derece olumlu katkılar sağladığı görülmektedir.

# İÇİNDEKİLER

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| ÖNSÖZ.....                                                               | ii  |
| İÇİNDEKİLER.....                                                         | iii |
| SİMGELER VE KISALTMALAR LİSTESİ.....                                     | 1   |
| TABLolar LİSTESİ.....                                                    | 4   |
| ŞEKİLLER LİSTESİ.....                                                    | 5   |
| ÖZET.....                                                                | 7   |
| BÖLÜM 1.                                                                 |     |
| GİRİŞ.....                                                               | 8   |
| BÖLÜM 2. LİTERATÜR TARAMASI                                              |     |
| 2.1.1 Yapay Zekâ.....                                                    | 10  |
| 2.1.2 Makine Öğrenmesi.....                                              | 10  |
| 2.1.2.1 Makine Öğrenmesi Türleri.....                                    | 11  |
| 2.1.2.1.1 Denetimli (Gözetimli) Öğrenme.....                             | 11  |
| 2.1.2.1.2 Denetimsiz (Gözetimsiz) Öğrenme.....                           | 11  |
| 2.1.2.1.3 Pekiştirmeli Öğrenme.....                                      | 11  |
| 2.1.3 Derin Öğrenme.....                                                 | 11  |
| 2.1.3.1 Yapay Sinir Ağları.....                                          | 12  |
| 2.1.3.2 Evrişimli Sinir Ağları (CNN).....                                | 12  |
| 2.2 Sağlık Alanında Yapay Zekânın Kullanımıyla İlgili Yapılan Çalışmalar | 14  |
| 2.3 Veri Gizliliği.....                                                  | 16  |
| 2.4 Dağıtık Sistemler.....                                               | 16  |
| 2.5 Federatif Öğrenme.....                                               | 18  |

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| 2.5.1 Federatif Öğrenme Türleri.....                                  | 21 |
| 2.5.1.1 Yatay Federatif Öğrenme (Horizontal FL).....                  | 21 |
| 2.5.1.2 Dikey Federatif Öğrenme (Vertical FL).....                    | 21 |
| 2.5.1.3 Karma Öğrenme (Federated Transfer Learning).....              | 22 |
| 2.5.2 Federatif Öğrenmeyle Alakalı Yapılan Çalışmalar.....            | 23 |
| <br>BÖLÜM 3. YÖNTEM                                                   |    |
| 3.1 Bakteri.....                                                      | 29 |
| 3.2 Kullanılacak Olan Veri seti.....                                  | 29 |
| 3.3 Veri Ön İşleme.....                                               | 29 |
| 3.3.1 Hiyerarşik Dizin Taraması.....                                  | 29 |
| 3.3.2 Format Standartizasyonu.....                                    | 29 |
| 3.3.3 Çözünürlük İndirgeme.....                                       | 30 |
| 3.3.4 Sıkıştırma ve Depolama.....                                     | 30 |
| 3.3.5 Hata Yönetimi.....                                              | 30 |
| 3.4 Eğitim Zamanlı Veri Çoğaltma ve Dönüşümler.....                   | 30 |
| 3.4.1 Mekânsal Değişmezliği Engellemek için Geometrik Dönüşümler..... | 31 |
| 3.4.2. Fotometrik Dönüşümler.....                                     | 31 |
| 3.4.3. Tensör ve Normalizasyon Dönüşümü.....                          | 32 |
| 3.5 Derin Öğrenme Mimarisi Seçimi.....                                | 32 |
| 3.5.1 Konvolüsyon Katmanı.....                                        | 33 |
| 3.5.2 Doğrusal Olmayan Katman.....                                    | 34 |
| 3.5.3 Havuzlama (Pooling) Katmanı.....                                | 34 |
| 3.5.4 Düzleştirme (Flattening) Katmanı.....                           | 35 |
| 3.5.5 Tam Bağlı (Fully Connected) Katman.....                         | 35 |
| 3.6 Derin Evrişimli Sinir Ağları.....                                 | 36 |

|                                                                                                             |    |
|-------------------------------------------------------------------------------------------------------------|----|
| 3.7 ResNet.....                                                                                             | 37 |
| 3.8 Transfer Öğrenmesi (Transfer Learning).....                                                             | 38 |
| 3.9 Federatif Öğrenme Mimarileri.....                                                                       | 40 |
| 3.9.1 Federatif Öğrenme Adımları.....                                                                       | 40 |
| 3.9.1.1 Global Modelin Başlatılması.....                                                                    | 40 |
| 3.9.1.2 İstemcilere Modelin Dağıtılması.....                                                                | 41 |
| 3.9.1.3 Yerel Eğitim Döngüsü.....                                                                           | 41 |
| 3.9.1.4 Güncellemelerin Sunucuya Geri Gönderilmesi.....                                                     | 42 |
| 3.9.1.5 Sunucuda Global Modelin Güncellenmesi.....                                                          | 43 |
| 3.9.1.6 Güncel Global Modelin Yeniden Cihazlara Dağıtımı                                                    | 43 |
| 3.9.2 Merkezi Federatif Öğrenme (CFL).....                                                                  | 44 |
| 3.9.3 Merkezi Olmayan Federatif Öğrenme (DFL).....                                                          | 44 |
| 3.9.4 Cross-Silo Federatif Öğrenme.....                                                                     | 45 |
| 3.9.5 Cross-Device Federatif Öğrenme.....                                                                   | 46 |
| 3.9.6 Hiyerarşik Federatif Öğrenme.....                                                                     | 47 |
| 3.9.7 Hibrid Federatif Öğrenme.....                                                                         | 47 |
| 3.9.8 Takviyeli Federatif Öğrenme.....                                                                      | 48 |
| 3.9.9 Güvenli Federatif Öğrenme.....                                                                        | 48 |
| 3.9.10 Projede kullanılacak olan mimari: Cross-silo.....                                                    | 48 |
| 3.10 Federatif Öğrenmede Kullanılan Yöntemler ve Algoritmalar.....                                          | 48 |
| 3.10.1 Federated Stochastic Gradient Descent (FedSGD).....                                                  | 48 |
| 3.10.2 Federated Averaging (FedAvg).....                                                                    | 49 |
| 3.10.3 Federated Proximal (FedProx).....                                                                    | 50 |
| 3.10.4 Federated Dynamics (FedDyn).....                                                                     | 50 |
| 3.10.5 Adaptive Federated Optimization Algorithms (Federatif Uyarlanabilir Optimizasyon Algoritmaları)..... | 51 |

|                                                                  |    |
|------------------------------------------------------------------|----|
| 3.10.6 Proje İçin Kullanılacak Olan Algoritmalar.....            | 51 |
| 3.11 Federatif Öğrenmede Kullanılan İletişim Protokolleri.....   | 52 |
| 3.11.1 Rest API.....                                             | 52 |
| 3.11.2 gRPC.....                                                 | 52 |
| 3.11.3 MQTT.....                                                 | 53 |
| 3.11.4 WebSocket.....                                            | 54 |
| 3.11.5 Proje İçin Kullanılacak Olan Protokol: gRPC.....          | 54 |
| 3.12 Federatif Öğrenmede Gizlilik ve Güvenlik Katmanları.....    | 55 |
| 3.12.1 Saldırı Yöntemleri.....                                   | 55 |
| 3.12.1.1 Membership Inference Attack.....                        | 55 |
| 3.12.1.2 Property Inference Attacks.....                         | 55 |
| 3.12.1.3 Model Poisoning Attacks.....                            | 55 |
| 3.12.1.4 Data Poisoning Attacks.....                             | 55 |
| 3.12.1.5 Gradient Inversion Attack.....                          | 56 |
| 3.12.2 Korunma Yöntemleri.....                                   | 56 |
| 3.12.2.1 Differential Privacy (DP).....                          | 56 |
| 3.12.2.2 Secure Aggregation.....                                 | 56 |
| 3.12.2.3 Homomorphic Encryption.....                             | 56 |
| 3.12.2.4 Trusted Execution Environments.....                     | 57 |
| 3.12.3 Proje Kapsamında Kullanılacak Olan Korunma Yöntemleri.... | 57 |

## BÖLÜM 4. UYGULAMAYA DAİR İLK ÇIKTILAR

|                                    |    |
|------------------------------------|----|
| 4.1 Uygulama ve Deney Ortamı.....  | 58 |
| 4.1.1 Jupyter Notebook.....        | 58 |
| 4.1.2 Google Colab.....            | 58 |
| 4.1.3 Kullanılan Kütüphaneler..... | 59 |

|                                                                                            |                |
|--------------------------------------------------------------------------------------------|----------------|
| 4.2. Deneysel Veri Yapılandırması ve Gizlilik Simülasyonu.....                             | 59             |
| 4.2.1 Veri Bölümleme Stratejisi.....                                                       | 59             |
| 4.2.2 İstemci Dağılımı ve Veri İzolasyonu.....                                             | 60             |
| 4.2.3 Gizlilik Protokolü ve Simülasyonu.....                                               | 60             |
| 4.2.4 Veri Artırma ve Doğrulama Stratejisi.....                                            | 61             |
| 4.3 Model Mimarisi ve Parametre Seçimi.....                                                | 61             |
| 4.3.1 ResNet18 Mimarisi ve Transfer Learning.....                                          | 61             |
| 4.3.2. Model Modifikasyonu ve Özelleştirme.....                                            | 62             |
| 4.3.3. FedAvg (Federated Averaging) Algoritması.....                                       | 62             |
| 4.3.4 Hiper-Parametre Konfigürasyonu.....                                                  | 63             |
| 4.4 Deneysel Bulgular ve Başarım Analizi.....                                              | 63             |
| 4.4.1 Eğitim Sonuçları.....                                                                | 64             |
| 4.4.2 Eğitim ve Doğrulama Sürecine Ait Kayıp ve Başarım Grafikleri.....                    | 65             |
| 4.4.3 Karmaşıklık Matrisi.....                                                             | 65             |
| <br>BÖLÜM 5. SONUÇLAR VE ÖNERİLER.....                                                     | <br>66         |
| <br>KAYNAKLAR.....                                                                         | <br>67         |
| EKLER.....                                                                                 | 75             |
| ÖZGEÇMİŞ.....                                                                              | 76             |
| <br>BSM 401 BİLGİSAYAR MÜHENDİSLİĞİ TASARIMI<br>DEĞERLENDİRME VE SÖZLÜ SINAV TUTANAĞI..... | <br><br><br>77 |

## SİMGELER VE KISALTMALAR LİSTESİ

|           |                                                                      |
|-----------|----------------------------------------------------------------------|
| ANN (YSA) | Yapay Sinir Ağları (Artificial Neural Networks)                      |
| API       | Uygulama Programlama Arayüzü (Application Programming Interface)     |
| BT        | Bilgisayarlı Tomografi                                               |
| CFL       | Merkezi Federatif Öğrenme (Centralized Federated Learning)           |
| CNN       | Evrişimli Sinir Ağları (Convolutional Neural Networks)               |
| cPDS      | Kümelenmiş İlkel Çift Ayrıştırma (Cluster Primal Dual Splitting)     |
| DFL       | Merkezi Olmayan Federatif Öğrenme (Decentralized Federated Learning) |
| DIBaS     | Bakteri Türlerinin Dijital Görüntüsü Veri Seti                       |
| DP        | Diferansiyel Gizlilik (Differential Privacy)                         |
| EEG       | Elektroensefalografi                                                 |
| EMR       | Elektronik Tıbbi Kayıtlar (Electronic Medical Records)               |
| FC        | Tam Bağlı Katman (Fully Connected Layer)                             |
| FDA       | Amerikan Gıda ve İlaç İdaresi (Food and Drug Administration)         |
| FedAvg    | Federatif Ortalama Algoritması (Federated Averaging)                 |
| FedDyn    | Federatif Dinamik Algoritması (Federated Dynamics)                   |
| FedProx   | Federatif Proksimal Algoritması (Federated Proximal)                 |
| FedSGD    | Federatif Stokastik Gradyan İniş                                     |
| FL        | Federatif Öğrenme (Federated Learning)                               |
| fPCA      | Federatif Temel Bileşen Analizi                                      |
| FTL       | Federatif Transfer Öğrenmesi (Federated Transfer Learning)           |
| gRPC      | Google Uzaktan Yordam Çağrısı (Google Remote Procedure Call)         |
| GPU       | Grafik İşlem Birimi (Graphics Processing Unit)                       |
| HHHFL     | Hiyerarşik Heterojen Yatay Federatif Öğrenme                         |
| HTTP      | Hiper Metin Transfer Protokolü                                       |
| IBM       | International Business Machines Corporation                          |

|                 |                                                                      |
|-----------------|----------------------------------------------------------------------|
| IID             | Bağımsız ve Özdeş Dağılmış (Independent and Identically Distributed) |
| IoT             | Nesnelerin İnterneti (Internet of Things)                            |
| KVKK            | Kişisel Verilerin Korunması Kanunu                                   |
| MAE             | Ortalama Mutlak Hata (Mean Absolute Error)                           |
| MEC             | Mobil Uç Bilişim (Mobile Edge Computing)                             |
| MIMIC-III       | Tıbbi Kayıtlar İçin Yoğun Bakım Bilgi Bankası                        |
| ML              | Makine Öğrenmesi (Machine Learning)                                  |
| MQTT            | Mesaj Kuyruğu Telemetri Taşıma Protokolü                             |
| MSE             | Ortalama Kare Hata (Mean Squared Error)                              |
| NLP             | Doğal Dil İşleme (Natural Language Processing)                       |
| RAM             | Rastgele Erişimli Bellek                                             |
| ReLU            | Doğrusal Olmayan Aktivasyon Fonksiyonu                               |
| ResNet          | Artık Ağlar (Residual Networks)                                      |
| RGB             | Kırmızı, Yeşil, Mavi Renk Uzayı                                      |
| RL              | Takviyeli Öğrenme (Reinforcement Learning)                           |
| RPC             | Uzaktan Adım Çağrısı (Remote Procedure Call)                         |
| SGD             | Stokastik Gradyan İniş (Stochastic Gradient Descent)                 |
| TEE / SGX       | Güvenilir Yürütme Ortamı (Trusted Execution Environment)             |
| TPU             | Tensor İşlem Birimi (Tensor Processing Unit)                         |
| $L(y, \hat{y})$ | Kayıp (Loss) fonksiyonu                                              |
| $C$             | Toplam sınıf sayısı                                                  |
| $y_i$           | i. sınıf için gerçek etiket                                          |
| $\hat{y}_i$     | i. sınıf için modelin tahmini olasılığı                              |
| $w$             | Model ağırlıkları                                                    |
| $g$             | Merkezi sunucuda hesaplanan ortalama / global gradyan                |
| $\eta$          | Öğrenme oranı (learning rate)                                        |
| $\nabla$        | Gradyan operatörü                                                    |
| $g_i$           | İstemci tarafında hesaplanan yerel gradyan                           |
| $n_i$           | i. cihazın sahip olduğu veri örneği sayısı                           |
| $n$             | Sistemdeki toplam veri örneği sayısı                                 |

|          |                                                           |
|----------|-----------------------------------------------------------|
| $K$      | Federatif ağıdaki toplam katılımcı cihaz / istemci sayısı |
| $\mu$    | FedProx algoritması için ceza katsayısı                   |
| $\alpha$ | FedDyn algoritması için düzenleme (regularization) gücü   |
| $h_i$    | Dinamik düzeltme terimi                                   |
| $F(x)$   | Artık (residual) eşleme fonksiyonu                        |

## TABLÖLAR LİSTESİ

|            |                                                              |    |
|------------|--------------------------------------------------------------|----|
| Tablo 1.1. | İş-Zaman Tablosu                                             | 9  |
| Tablo 3.1. | 56 Katmanlı CNN – 20 Katmanlı CNN                            | 37 |
| Tablo 3.2. | ResNet                                                       | 38 |
| Tablo 3.3  | Transfer Öğrenmesi                                           | 39 |
| Tablo 3.4. | Federatif Öğrenme-2                                          | 41 |
| Tablo 3.5. | Federatif Öğrenme-3                                          | 42 |
| Tablo 4.1. | Hiper-Parametre Konfigürasyonu Tablosu                       | 63 |
| Tablo 4.2. | Eğitim Sonuçları                                             | 64 |
| Tablo 4.3. | Eğitim ve Doğrulama Sürecine Ait Kayıp ve Başarım Grafikleri | 65 |
| Tablo 4.4. | Karmaşıklık Matrisi                                          | 65 |

## ŞEKİLLER LİSTESİ

|             |                                                                 |    |
|-------------|-----------------------------------------------------------------|----|
| Şekil 2.1.  | Yapay Sinir Ağları                                              | 12 |
| Şekil 2.2.  | Evrişimli Sinir Ağları                                          | 13 |
| Şekil 2.3   | Yapay zekâ, makine öğrenmesi ve derin öğrenme arasındaki ilişki | 13 |
| Şekil 2.4.  | Dağıtık Sistem Çeşitleri                                        | 17 |
| Şekil 2.5.  | Dağıtık Sistemlere Yapılabilecek Saldırıları                    | 18 |
| Şekil 2.6.  | Federatif Öğrenme                                               | 19 |
| Şekil 2.7.  | Federatif Öğrenme                                               | 20 |
| Şekil 2.8.  | Yatay Federatif Öğrenme                                         | 21 |
| Şekil 2.9.  | Dikey Federatif Öğrenme                                         | 22 |
| Şekil 2.10. | Karma Öğrenme                                                   | 23 |
| Şekil 3.1.  | DIBas veri seti                                                 | 29 |
| Şekil 3.2.  | Tensör ve Normalizasyon Dönüşümü                                | 32 |
| Şekil 3.3.  | Evrişimli Sinir Ağları                                          | 33 |
| Şekil 3.4.  | Konvolüsyon Katmanı                                             | 34 |
| Şekil 3.5.  | Havuzlama Katmanı                                               | 35 |
| Şekil 3.6.  | Düzleştirme Katmanı                                             | 35 |
| Şekil 3.7.  | Derin Evrişimli Sinir Ağları                                    | 36 |
| Şekil 3.8.  | ResNet18 Mimarisi                                               | 38 |
| Şekil 3.9.  | Transfer Öğrenmesi-2                                            | 39 |
| Şekil 3.10. | Güncel Global Modelin Yeniden Cihazlara Dağıtımı                | 43 |
| Şekil 3.11. | Centralized Federated Learning (DFL)                            | 44 |
| Şekil 3.12. | Decentralized Federated Learning (CFL)                          | 45 |
| Şekil 3.13. | Cross-silo Federated Learning                                   | 46 |

|             |                                                            |    |
|-------------|------------------------------------------------------------|----|
| Şekil 3.14. | Cross-device Federated Learning                            | 46 |
| Şekil 3.15. | Hierarchical Federated Learning                            | 47 |
| Şekil 3.16. | Tüm Federatif Öğrenme Topolojileri ve Hesaplama Yöntemleri | 48 |
| Şekil 3.17. | Rest API akış şeması                                       | 52 |
| Şekil 3.18. | gRPC'nin akış şeması                                       | 53 |
| Şekil 3.19. | MQTT'nin akış şeması                                       | 53 |
| Şekil 3.20. | WebSocket'in akış şeması                                   | 54 |

## ÖZET

**Anahtar kelimeler:** Federatif Öğrenme, Derin Öğrenme, Veri Gizliliği, Bakteri Sınıflandırma, Sağlıkta Yapay Zekâ

Sağlık sektöründe veri gizliliğinin kritik bir öneme sahip olduğu ve kişisel verilerin korunmasının yasal zorunluluklarla sınırlandırıldığı günümüzde, Federatif Öğrenme (Federated Learning) yaklaşımı, verilerin merkezi bir sunucuda toplanmasına gerek kalmadan işbirlikçi model eğitime olanak tanıyan dağıtık bir makine öğrenmesi teknolojisi olarak tanımlanmaktadır. Bu yöntem sayesinde, farklı kurumlarda bulunan hassas tıbbi veriler yerel cihazlarda işlenerek, yalnızca model parametrelerinin veya ağırlıklarının paylaşılması yoluyla ortak ve güçlü bir yapay zekâ modeli geliştirilebilmektedir. Ham verinin paylaşılmadığı bu yapı, veri mahremiyetini yasalara uygun şekilde korurken büyük veri analitiğinin gücünden yararlanılmasını sağlayan güvenli bir altyapı sunmaktadır.

Bu tasarım çalışması ile DIBaS veri seti kullanılarak ResNet18 mimarisi ve FedAvg algoritması temelinde çalışan bir bakteri sınıflandırma sistemi geliştirilmiş olup, Cross-silo mimarisinin kurumsal sağlık ağlarında veri gizliliğini ihlal etmeden yüksek başarılı modeller üretebileceği ortaya konmuştur. Çalışma sonucunda %97.95 oranında eğitim başarıımı elde edilmiş; gRPC protokolü ve güvenli birleştirme (Secure Aggregation) tekniklerinin kullanılmasıyla iletişim maliyetlerinin düşürüldüğü ve veri güvenliğinin sağlandığı görülmüştür. Ayrıca, bu tür dağıtık sistemlerde modelin tersine çevrilmesi gibi saldırılara karşı Diferansiyel Gizlilik (Differential Privacy) gibi ek güvenlik katmanlarının entegre edilmesi ve kimlik doğrulama mekanizmalarının etkinleştirilmesi önerilmiştir.

## BÖLÜM 1. GİRİŞ

Sağlık sektörü her geçen gün gelişmektedir. Teknolojinin de gelişmesiyle bu gelişim, giderek hızlanmıştır. Önceden çok zor tedavi edilen hastalıklar günümüzde teknolojinin yardımıyla çok daha kolay bir şekilde tedavi edilir hale gelmiştir.

Bununla beraber hastalıkların teşhisi de kolaylaşmıştır. Özellikle yapay zekanın gelişmesiyle beraber günler süren hastalık teşhisleri dakikalar içinde konulabilmektedir. Bu sayede hastalığın daha fazla ilerlemeden önünü kesmek mümkün olabilmektedir. Bu noktada yaptığımız proje, hastalık teşhisi konusuna yardımcı olmaktadır.

Bu proje yapay zekayı kullanarak laboratuvardaki mikroskop görüntülerinden bakteri sınıflandırması yapmayı amaçlamaktadır.

Fakat burada şöyle bir sorun göze çarpmaktadır. Bildiğiniz gibi yapay zekâ modelleri, öğrenebilmek için çok fazla veriye ihtiyaç duyar. Bununla beraber verilerin çeşitliliği de modelin genelleme yeteneğini artırma açısından önem arz etmektedir.

Bunun yanı sıra bu modeli oluşturmak için gerekli verilere erişimin de sağlanması gerekmektedir. Fakat sağlık sektöründe bu durum, diğer sektörlerle kıyasla çok daha zordur. Çünkü ülkemizdeki KVKK'ye göre sağlık verisi, “özel nitelikli kişisel veri” kategorisindedir. Bu yüzden hangi amaç olursa olsun hasta verileri paylaşılamaz. Bizim projemizde federatif öğrenme kullanıldığı için bu bir sorun teşkil etmemektedir.

Federatif öğrenme, birden fazla cihazın veya kurumun verilerini merkezi bir sunucuya göndermeden, ortak bir yapay zekâ modeli eğitmesini sağlayan dağıtık bir makine öğrenmesi yöntemidir. Bunu sunucudaki global modeli, istemcilere göndererek yapar. İstemciler, modelleri birkaç epoch boyunca yerel olarak eğitir. Bütün istemcilerden gelen güncel ağırlıklar, sunucuya aktarılır. FedAvg (federated average) veya veri dengesizliği durumlarında FedProx (federated proximal) gibi algoritmalar kullanılarak yeni global model oluşur. Bu model, tekrardan istemcilere dağıtılarak süreç tekrarlanır. Bu öğrenme yöntemi sayesinde veriler paylaşılmadan model eğitilebildiği için, gizlilik yasalarına uygun biçimde yapay zekâ modelleri geliştirilebilir.

Proje için yapılacak işler, aylara bölünmüştür. Bu süre içerisinde iş paketleri, bölümlere ayrılmıştır. Tablo 1.1’de görüleceği gibi iş paketlerinin süresi ve sıralaması belirlenmiştir.

| Sorumlu Kişi                                   | Yapılacak İşler                         | Aylar |       |        |
|------------------------------------------------|-----------------------------------------|-------|-------|--------|
|                                                |                                         | Ekim  | Kasım | Aralık |
| İsmail Alper Karadeniz<br>Muhammed Yusuf Öngel | Literatür Taraması                      |       |       |        |
| Muhammed Yusuf Öngel                           | Kavramsal Tasarım                       |       |       |        |
| İsmail Alper Karadeniz                         | Verilerin Toplanması ve Ön İşleme       |       |       |        |
| İsmail Alper Karadeniz<br>Muhammed Yusuf Öngel | Modelin ve Algoritmaların Araştırılması |       |       |        |
| İsmail Alper Karadeniz<br>Muhammed Yusuf Öngel | Federatif Öğrenme Tasarımı              |       |       |        |
| İsmail Alper Karadeniz<br>Muhammed Yusuf Öngel | Modelin ve Algoritmaların Uygulanması   |       |       |        |
| İsmail Alper Karadeniz<br>Muhammed Yusuf Öngel | Oluşan Prototipin Test Edilmesi         |       |       |        |

Tablo 1.1 İş-Zaman Tablosu

## **BÖLÜM 2. LİTERATÜR TARAMASI**

### **2.1.1 Yapay Zekâ**

Yapay zekânın tarihini kısaca özetlersek 1940'lı yıllarda Mathison Turing “Makineler düşünebilir mi?” sorusunu sorarak yapay zekâ hakkındaki ilk adımlardan birini atmıştır. McCarthy 1956'lı yıllarda yapay zekâ adını ortaya atmıştır ve 1974 yılına kadar yapay zekâda çok fazla çalışma yaşanmıştır.

Ardından 1974'ten 1980 yılına kadar durgun bir dönem olmuştur. Bu durgunluğun sebebi sistem gereksinimlerinin karşılanamaması ve projelerdeki başarısızlık sebebiyle yatırımın düşmesidir. Bu yüzden yapay zekâ ile ilgili araştırmalar ve diğer projeler fon bulmakta zorlanmıştır.

Yapay zekâ araştırmaları, 1980'li yılların ortalarında sistemlerin de daha fazla gelişmesiyle yeniden canlandı. Fakat sistemler verilere çok bağımlıydı. Veri kümelerinin yetersiz oluşu ve 1980'lerde depolama maliyetlerinin çok fazla olması nedeniyle yeniden durgunluk dönemine girildi.

Günümüzde bu tür sorunların çoğu giderilmiştir. Veri kümeleri artık çok daha fazla, verilere erişim çok daha kolaylaşmıştır. Depolama sistemleri, gelişerek daha az maliyetle depolama sağlamıştır. Böylelikle yapay zekânın gelişimi hız kazanmıştır [1].

### **2.1.2 Makine Öğrenmesi**

Makine öğrenmesi, bilgisayarların insanlar gibi öğrenmesini sağlamak amacıyla çeşitli algoritmaların ve tekniklerin geliştirildiği bir çalışma alanıdır.

İlk olarak 1952 yılında, Arthur Samuel tarafından tasarlanmıştır. IBM'in Poughkeepsie'deki laboratuvarında dama oynaması için geliştirilmiştir. Arthur Samuel, bu

programın oynadığı her dama oyununda kendini biraz daha geliştirmesi, hatalarını tespit edip düzeltmesi ve elde ettiği bu verileri kullanarak oyunu kazanmanın daha güçlü yollarını bulmasını amaçlamıştır. Bu oto-öğrenme programı, makine öğrenmesinin ilk örneklerinden biridir [2,3].

### **2.1.2.1 Makine Öğrenmesi Türleri**

#### **2.1.2.1.1 Denetimli (Gözetimli) Öğrenme (Supervised Learning)**

Eğer veri setinde beklenen çıktılar varsa bu durum denetimli (gözetimli) öğrenme olarak ifade edilir. Yani bir modeli, belirli bir girdi için "doğru" çıktıyı tahmin edecek şekilde eğitir. Örnek olarak yapılan reklamlar sonucu elde edilen satışlar verilebilir.

#### **2.1.2.1.2 Denetimsiz (Gözetimsiz) Öğrenme (Unsupervised Learning)**

Eğer veri setinde beklenen çıktılar yoksa bu durum gözetimsiz öğrenme problemi olarak geçer. Yani bir modeli, verilerdeki bağımlılıkları ve korelasyonları ayırt edecek şekilde eğitir. Denetimli öğrenmeden farklı olarak çıktılarının karşılaştırılacağı herhangi bir doğru çıktı içermez. Örnek olarak bir ülkenin bölgeleri özelindeki suç oranları verilebilir.

#### **2.1.2.1.3 Pekiştirmeli Öğrenme (Reinforcement Learning)**

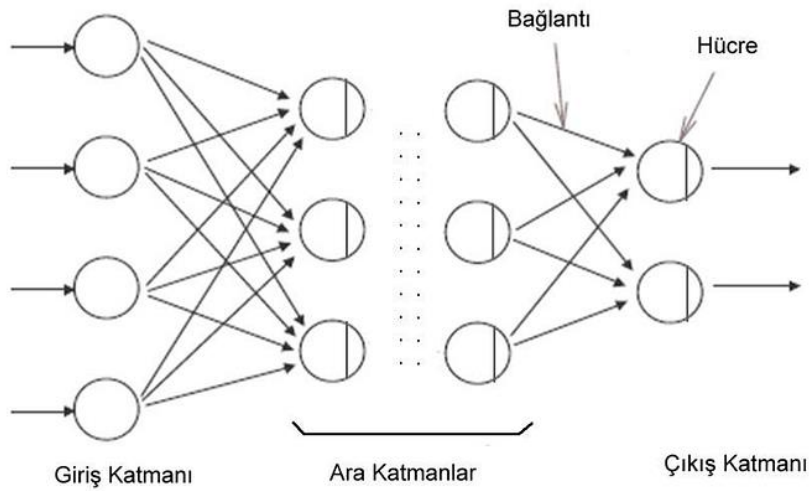
Deneme yanılma yoluyla yapılan öğrenme pekiştirmeli öğrenmedir. Yanlış hamlelerden ders çıkararak öğrenen bu modeller, durumu değerlendirir ve en doğru hamleyi yapacak şekilde kendini eğitir. Dama oynamayı öğrenen ve her oyunda hatalarından ders çıkarıp damada uzmanlaşan bir program, pekiştirmeli öğrenmeye örnek olarak verilebilir. [3]

### **2.1.3 Derin Öğrenme**

Derin öğrenme, bir makine öğrenme yöntemlerinden biridir. Verilen bir veri kümesi kullanarak çıktıları tahmin edecek bir yapay zekâ eğitimi yapmamızı sağlar. Yapay zekâyı eğitmek için yukarıda bahsedilen öğrenme yöntemleri kullanılabilir. [4]

### 2.1.3.1 Yapay Sinir Ağları

Yapay sinir ağları (YSA) bir derin öğrenme yöntemidir. Büyük veri kümelerinden öğrenme ve karmaşık öznitelik çıkarım yeteneğine sahiptir. Birbirine bağlı nöron katmanlarından oluşur. Nöronlar arasında ileri ve geri yayılım sağlayarak kayıp oranını minimuma indirmek istenir. Geri yayılım esnasında ağırlıklar güncellenir. [5]



Şekil 2.1 Yapay Sinir Ağları

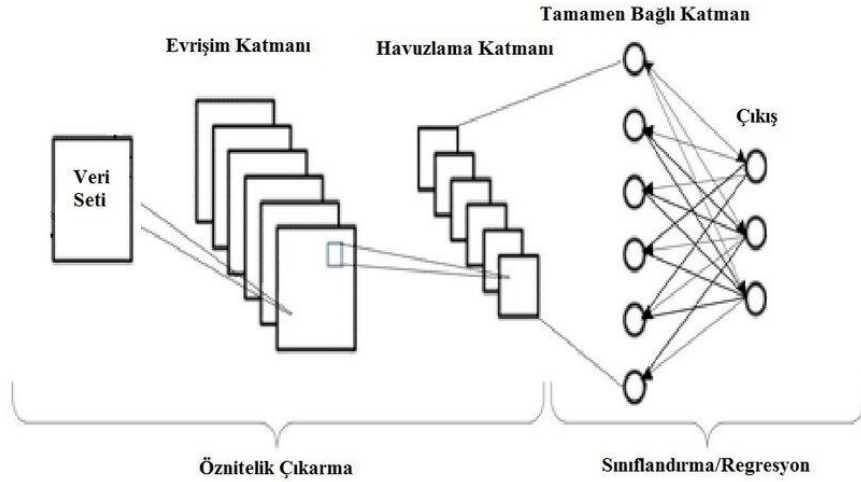
Kısaca üç katmandan oluşur. Giriş katmanı giriş verilerini alır. Ara katmanlar girdilerimizde matematiksel hesaplamalar yapar. Bunun için çeşitli algoritmalar geliştirilmiştir. Çıktı katmanı, çıktı verilerini döndürür.

### 2.1.3.2 Evrişimli Sinir Ağları (CNN)

Evrişimli Sinir Ağları (CNN), vücudumuzdaki görme sisteminden ilham alarak tasarlanmış yapay sinir ağlarından biridir. Özellikle görsel verileri analiz etmek için oldukça uygundur.

CNN'ler nesneleri tanımlamada çok etkili oldukları için, görüntü tanıma ve nesne tanıma gibi bilgisayarlı görü (computer vision) görevlerinde sıklıkla kullanılırlar. Yaygın olarak otonom araçlarda, yüz tanıma ve tıbbi görüntü analizinde kullanılır.

Giriş verisinin özelliklerini çıkarmak için evrişim ve havuzlama gibi katmanlar kullanılır. *Evrişim katmanları*, görüntüdeki filtrelerin kaydırılmasını sağlayarak yerel özelliklerin tespit edilmesini sağlar. *Havuzlama katmanları*, özellik haritalarını küçültür ve hesaplama maliyetini azaltır. *Tam bağlantılı katmanlar*, özellik haritalarını düzleştirir ve bu bilgiyi sınıflandırma veya regresyon amacıyla kullanır. ReLU gibi aktivasyon fonksiyonları sıkça tercih edilir çünkü modellere doğrusallık katar. [6]



Şekil 2.2 Evrişimli Sinir Ağları

Derin öğrenme, makine öğrenmesi ve yapay zekâ kavramlarının arasındaki ilişki şu şekilde gösterilebilir:



Şekil 2.3 Yapay zekâ, makine öğrenmesi ve derin öğrenme arasındaki ilişki

## 2.2 Sağlık Alanında Yapay Zekânın Kullanımıyla İlgili Yapılan Çalışmalar

Yapay zekanın gelişimiyle birlikte artık yapay zeka sağlık sektöründe de önemli bir yere sahiptir. Geçtiğimiz yıllarda yapılan araştırmalar, tıpta yapay zekânın piyasa değerinin 2021 yılına kadar 6,6 milyar dolara ulaşacağını, 2026 yılına kadar ise 45,2 milyar dolara ulaşacağını tahmin ediyordu. Fakat COVID-19 pandemisiyle birlikte 2026 yılında beklenen piyasa değeri 291,5 milyar dolara kadar çıkmıştır. [7]

Yapay zekâ, sağlık yönetiminde de var olabilir. Genel sağlık, belgelendirme, maliyet ve kalite gibi alanların yönetiminde de kendine rahatlıkla bir yer bulabilir. Amerika Birleşik Devletleri'ndeki bir enstitü olan Centerstone Araştırma Enstitüsü tarafından yapılan çalışmada yapay zekâ teknolojisi ile hastalara tanı koyma yöntemlerinin, geleneksel tanı koyma yöntemlerine göre maliyet olarak çok daha düşük olduğu görülmüştür. Çalışma kapsamında, rastgele seçilen yaklaşık 500 hastanın fiziksel bulguları ve hastalıklarına ait sonuçları yapay zekâ algoritmaları ile kıyaslanmıştır. Yapılan çalışmaya göre, birim başına düşen maliyetlere bakıldığında anlamlı farklar olduğu ortaya konulmuştur. Yapay zekâ uygulamaları kullanılarak yapılan incelemeler, enstitüye ortalama 189 dolar maliyet çıkarırken, geleneksel yöntemler kullanılarak yapılan incelemelerde maliyetin ortalama 497 dolar olduğu görülmüştür. [8]

IBM tarafından tasarlanan IBM Watson, hekimlerin hastalarına ait elektronik sağlık verilerini incelemelerine, bir hastalıkla ilgili akademik yayınlara erişmeye ve hastalıkla ilgili dokümanları incelemelerine yardımcı olan bir yapay zeka sistemidir. Hindistan'da yapılan bir araştırmaya göre, Watson onkoloji sistemi kullanılarak meme kanseri olan 638 hastaya önerilen tedavi tavsiyelerinin, bu alanın uzmanı olan kurulun önerdiği tedaviler ile %90 oranında benzerlik olduğu ve zamandan büyük ölçüde tasarruf edildiği görülmüştür. Bu kapsamda araştırmacılar, birtakım öneriler oluşturmak için verileri toplamak ve analiz etmek için manuel yöntem ve Watson sistemini kullanmış ve ikisinde de geçen süreyi karşılaştırmışlardır. Manuel yaklaşım ile ortalama 20 dakika (hekimler bu yapmaya devam ettikçe bu süre 12 dakikaya kadar düşmüştür), Watson onkoloji sistemi ile ise yalnızca 40 saniye sürdüğü görülmüştür [9].

Bilgisayar destekli görüntü teknolojileri, radyoloji alanında büyük bir öneme sahiptir. Çünkü tanı süreçlerini olabildiğince hızlandırmıştır. Radyologların bir milyondan fazla hastayı tedavi etmesine yardımcı olan Aidoc, görüntü analizi için oluşturulmuş bir yapay zekadır. Veriler tamamen bulut bilişim içinde tutulur. Hastanelerde görüntülenme cihazı dışında herhangi bir cihaza gerek yoktur. [66]

Yine IoT uygulamaları kullanılarak kronik hastalıklara erken tanı konulabilmektedir. Yapay zeka destekli bir IoT uygulamasıyla kullanıcının verileri makine öğrenmesi ile işlenerek kişinin sağlıksal durumu üzerinde tahmin yapıp risk varsa bu konuda bir uyarı verme görevine sahiptir. [67]

Sağlıkta dijital dönüşüm konusuna önem veren Kore, hastalara ait verileri ve gözlem sonuçlarının tamamını bulutta tutuyor. Bu veriler doktor kontrol etmeden önce CardioLogs gibi FDA onaylı yapay zekâ destekli programlardan geçiyor. Bu programlara hastalık teşhisine de yardımcı oluyor ve en sonunda sonuçlar doktorlara sunuluyor. Böyle hastanın teşhis süreci, normal bir sürece göre daha hızlı işlemiş oluyor. [68]

Yapılan bir diğer araştırmada, 10 adet gelecek vadeden yapay zekâ uygulaması ABD’de 2026’ya kadar kullanılırsa yıllık yaklaşık olarak 150 milyar dolar tasarruf sağlayabileceği görülmüştür. Bu kapsamda, en çok tasarruf sağlayan alan 40 milyar dolarlık tasarruf ile robotik cerrah sistemleri olmuştur. Ondan sonra sırasıyla sanal hemşirelik asistanları 20 milyar dolar, ilaçların doğru doz tespiti 16 milyar dolar, radyolojik görüntüler üzerinden hastalık tespiti ise 3 milyar dolar tasarruf ettirebileceği ortaya konmuştur. [10]

Yapay zeka tıp eğitime de doğrudan katkı sunuyor. Doğal dil işleme tekniği de kullanılarak öğrencilerin sorduğu sorulara anında cevap verebiliyor. Yapılan bir çalışmada, yapay zeka destekli bir sohbet robotu kullanılması durumunda bir tıp öğrencisinin klinik karar alma alıştırmaları hakkında anında geri bildirim almasına yardımcı olduğunu göstermiştir. [11,12]

Çin’de yapılan bir yapay zeka uygulaması, 19 yaşında komada olan bir genç üzerinde uygulanmış ve yapay zekadan uyanıp uyanmayacağını tahmin etmesi istenmiş. Yapay zeka ise doktorların aksine gencin uyanacağını söylemiş ve başarılı olmuştur. Uyandıktan sonra yapılacak olanları da söyleyerek kaynakları doğru ve zamanında kullanılmasına katkıda bulunmuştur. [13]

Teksa’sta bulunan Metodist Araştırma Enstitüsünde meme kanserini tespit eden bir yapay zekâ uygulaması geliştirilmiştir. Bu kapsamda bu yapay zekayı, 500 hastaya ait mamografi ve patoloji raporlarını değerlendirmek için kullanmışlardır. Bunun sonucunda yapay zeka, sonuçları %99 oranla doğru tahmin etmiştir ve bir hekimden 30 kat daha hızlı bir şekilde teşhis etmeyi başarmıştır [14].

İrlanda’da bulunan Özel Mater Hastanesi, radyoloji birimiyle ilgili sorunlara bir cevap bulmak için yapay zeka teknolojisi yardımıyla hastanenin bir dijital ikizini oluşturmuşlar. Yapay zekanın verdiği tavsiyeler sonucunda BT çekimi için hasta bekleme süresinde 13 dakika, MRI (Magnetic Resonance Imaging) çekimi için 25 dakika azalma sağlanmıştır. Aynı zamanda MRI kapsamında her gün 50 dakika fazla mesai yapılarak, personel giderleri azaltılmış ve yıllık 9500 Euro tasarruf elde edilmiştir [15,16].

## **2.3 Veri Gizliliği**

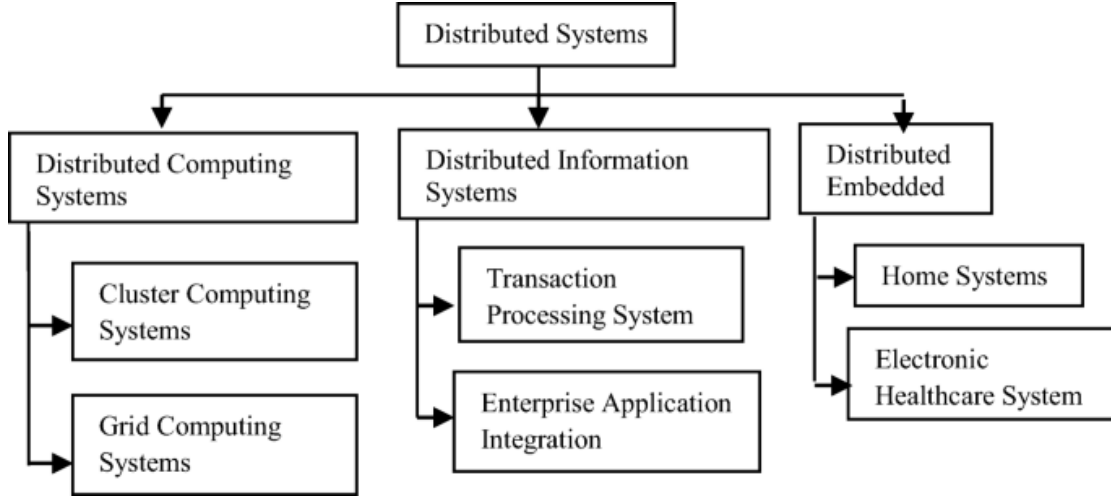
Sağlık alanında artan büyük veri miktarı gizlilik problemlerine sebep olmaktadır. Elektronik sağlık kayıtları, giyilebilir sağlık cihazları ve çeşitli kaynaklardan gelen sağlık verileri yüksek oranda kişisel ve özel bilgiler içermektedir. Bu durum sağlık verilerinin korunmasını ayrıca önemli hale getirmektedir.

Yapay zekâ tabanlı teknolojiler büyük miktarda veriye ihtiyaç duymaktadır. Bu veriler paylaşılmadan önce zorunlu olarak anonimleştirilse de bazı algoritmalar anonim veri üzerinden gruplandırma yaparak bazı kişisel bilgileri tahmin edebilmektedir. Ayrıca veri akışına dayalı merkezi öğrenme yöntemleri ile kurulan sistemler hassas olabilmektedir. Bu sistemler saldırganların kişisel sağlık verilerini çalmasına, değiştirmesine veya silmesine sebep olabilir. Bu durum hasta sağlığını ciddi bir şekilde tehlikeye atabilir. Saldırıya maruz kalan kuruma duyulan güveni azaltabilir. [17]

Hindistan’da bir sağlık kuruluşunda üst seviye güvenlik önlemlerine sahip olunmasın rağmen hasta kayıtları, randevular ve benzeri idari sistemler ile ilişkili verileri hedef alan bir siber saldırı sonucu sistem işlevsiz kalmıştı. Bu hizmetler 2 haftadan fazla süre ile manuel olarak işletilmek zorunda kaldı ve 30 milyondan fazla verinin tehlikede olduğu düşünülmektedir [18]. Saldırganların dijital insülin pompalarının çalışmasını engellediği başka bir örnekte mevcuttur [19].

## **2.4 Dağıtık Sistemler**

Dağıtık sistem, fiziksel olarak ayrı ama mantıksal olarak ağ vasıtasıyla bağlı ve iletişim halinde olan cihazların oluşturduğu homojen gözüken yapıdır.



Şekil 2.4 Dağıtık Sistem Çeşitleri

Bu yapıda düğümler kaynak paylaşımı yapar ve takım halinde çalışır. Düğümler birbirinden bağımsızdır ve birbirlerinin çalışmasını etkilemezler. Dağıtık sistemlerin temel amacı kullanıcılar veya uygulamalar tarafından uzak mesafelere erişim sağlamayı basitleştirmektir. [61]

Dağıtık sistemler sistemin işlem kapasitesini artırmak için mevcut sunucuları daha işlevsel yapmak yerine daha fazla sunucu ekleyebilme avantajı vardır.

Altyapı maliyetleri için verimlilik ve yüksek kullanılabilirlik sağlarlar [20].

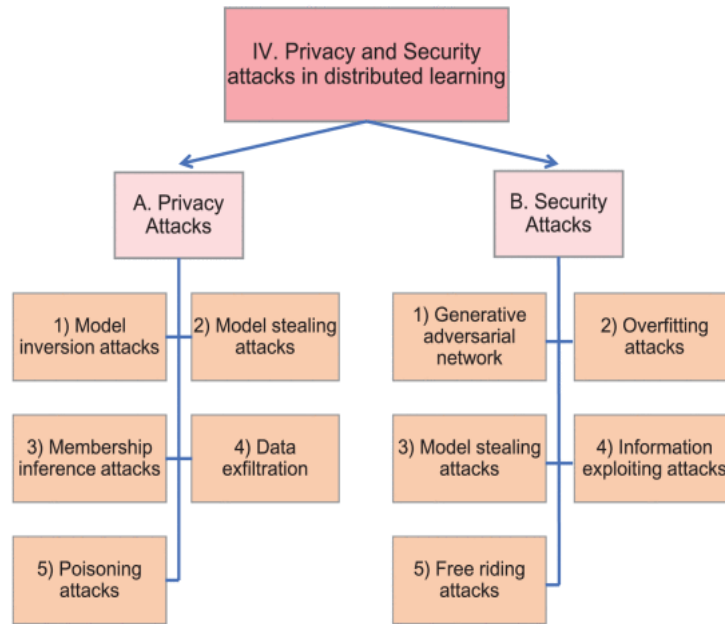
Günümüzde sağlık alanında üretilen veriler gelişen teknolojinin etkisiyle katlanarak artmakta ve geleneksel veri işleme yöntemleri giyilebilir cihazlardan genom dizilemeden ve tıbbi görüntüleme gibi yapılandırılmış ve özelleşmiş karmaşık verileri işleme konusunda yetersiz kalmaktadır. Dağıtık sistemler ve büyük veri entegrasyonu bu sorunu çözmek için temel bir yol haline geldi ve geleceğe dönük etkili bir yol olarak karşımıza çıkmaktadır. Dağıtık sistemler yüksek veri ile eğitilen tek bir merkezi modele göre çok daha hızlı ve verimli bir yaklaşımdır. Dağıtık sistem mimarileri yüksek miktarda veri ile çalışabilecek altyapıya sahiptir [21,22].

Amerikan Tıp Bilişimi derneği Dergisi'nde yayınlanan çok önemli bir çalışma dağıtık hesaplamaların Kuzey Amerika'daki 47 sağlık kuruluşunda nasıl uygulandığını inceledi. Bulguları dağıtık sistem uygulayan tesislerin geleneksel yöntemlere kıyasla veri işleme süresinde %32 azalma ve hesaplama verimliliğinde %41 iyileşme olduğunu ortaya koydu [23].

Dağıtık sistemlerin bu avantajı ve esnek yapısının yanı sıra gizlilik, güvenlik, algoritmik adalet gibi dikkat edilmesi gereken bazı kritik noktalar ve zayıf yanları da mevcuttur [24,25].

Dağıtık sistemlerin dezavantajı olarak karmaşıklığın artması, sistemin güvenliğini sağlamanın zorlaşması ve saldırılara açık hale gelmesi, hata ayıklama zorlukları söylenebilir [20,23].

Dağıtık sistem alt yapısını kullanarak sağlanan dağıtık öğrenmede gizlilik ve güvenlik saldırıları aşağıda gösterilmiştir.



Şekil 2.5 Dağıtık Sistemlere Yapılabilecek Saldırıları

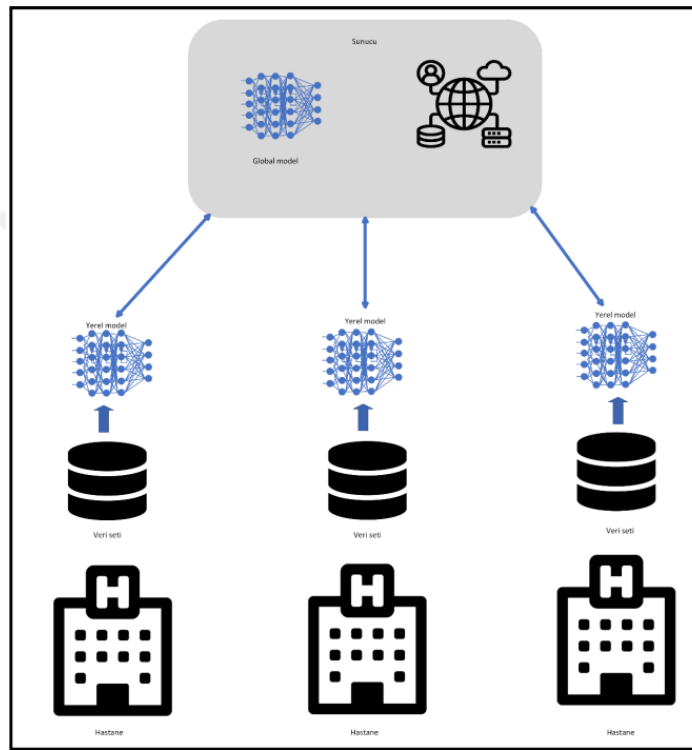
## 2.5 Federatif Öğrenme

Veri gizliliği ve güvenliğini sağlamak amacıyla Google Yapay Zeka ekibi tarafından ilk olarak 2016 yılında önerilen federatif öğrenme, sadece veri gizliliğini sağlamakla kalmıyor. Bunun dışında birçok avantajı da beraberinde getiriyor.

Federatif öğrenme, birden fazla cihazın veya kurumun verilerini merkezi bir sunucuya göndermeden, ortak bir yapay zekâ modeli eğitmesini sağlayan dağıtık bir makine öğrenmesi

yöntemidir. [63] Dağıtık sistemin amacı işlem aşamasını hızlandırmakken federatif öğrenmenin amacı, veri sızıntısı olmadan model geliştirmesi yapmaktır. Bunu sunucudaki global modeli, istemcilere göndererek yapar. İstemciler, modelleri birkaç epoch boyunca yerel olarak eğitir. Bütün istemcilerden gelen güncel ağırlıklar, sunucuya aktarılır. Farklı algoritmalar kullanarak merkez sunucuda global bir model oluşturur.

Federatif öğrenme, özellikle tıp alanındaki hastaların verilerini gizleme konusunda dikkat çeker. Hastaların verileri, mahremiyet ve güvenlik açısından son derece hassastır. Bu konuda ülkemizdeki KVKK kanununa göre “*Özel nitelikli kişisel verilerin işlenmesi yasaktır [26]*” der. Bunun dışında bu verilerin merkezi sunucularda toplanması ciddi gizlilik riskleri oluşturabilir. Federatif öğrenme, verileri hiçbir yerde depolamaz, onların ham verilerini değil ağırlıklarını işleyerek bir model oluşturur.

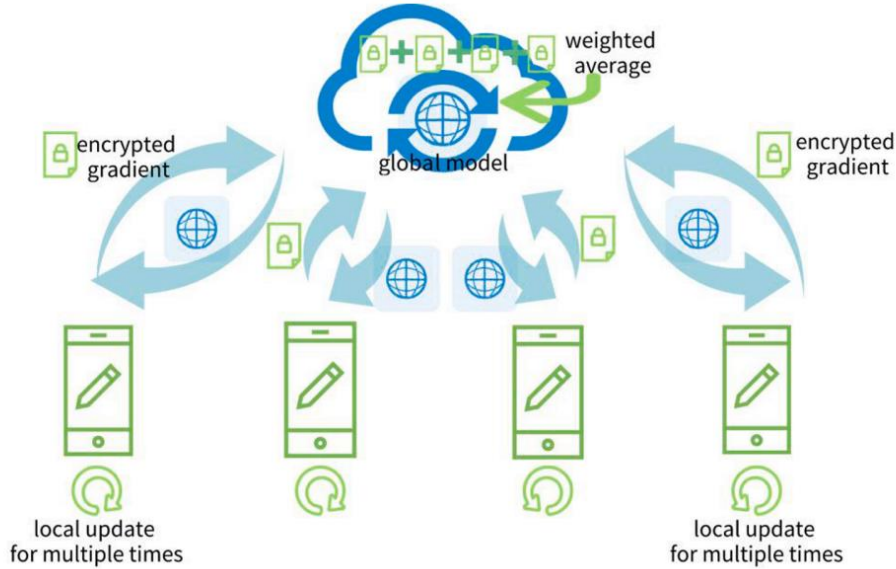


Şekil 2.6 Federatif Öğrenme

Bunun dışında federatif öğrenmenin çok daha farklı avantajları vardır. Bunlardan birisi de bilgilerin farklı yerlerden alınmasıdır. Bu sayede farklı yerlerdeki farklı durumlara da çözüm üretebilen bir yapay zekâ modeli ortaya çıkabilir.

Federatif öğrenmenin, dağıtık sistemlere göre de avantajları vardır. Bu avantajlardan biri de verilerin her bir cihazdaki yapısının farklı olmasıdır. Dağıtık sistemlerin amacını söylemiştik. Bu durumda veriler genelde dengeli ve benzer dağılır. Fakat federatif öğrenmede verilerin her bir cihazdaki yapısı farklıdır (Non-IID).

Bir diğer avantajı ise merkezi sunucunun yönetici rolünde olmamasıdır. Dağıtık sistemlerde merkezi sunucu; veriyi toplar, dağıtır ve sonuçları kontrol eder. Bu yapı, veri aktarımı, depolama ve bant genişliği açısından önemli bir maliyet oluşturur. Fakat federatif öğrenmede veriyi cihazlar tutar. Kendi modelini eğitir yalnızca model parametrelerini merkezi sunucuda paylaşır. Böylece merkezî sunucunun rolü azaltılarak sistem yarı-merkezsiz (decentralized) bir yapı kazanır ve veri gizliliği korunurken iletişim maliyetleri düşürülür.



Şekil 2.7 Federatif Öğrenme

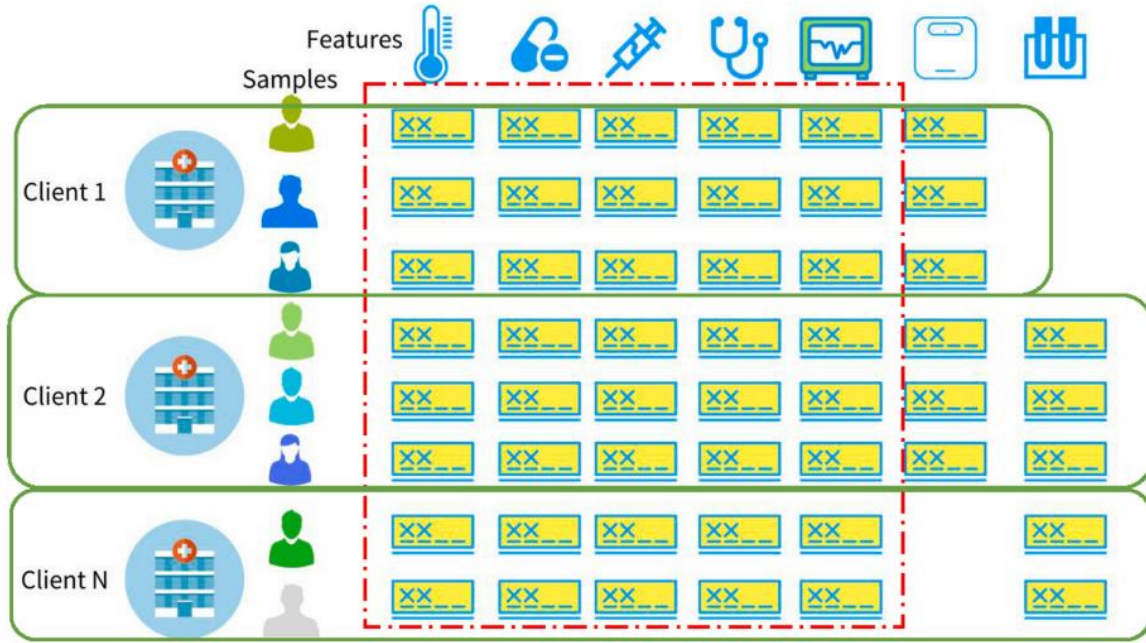
Dağıtık sistemlerdeki bir diğer problem, verilerin ağırlıklarının eşit olarak değerlendirmesidir. Bu durum, belli bir yerden gelen verilerin daha fazla olması durumunda modelin doğruluk oranını büyük oranda manipüle edebilir. Federatif öğrenmede her bir cihazın gönderdiği model, o cihazın ağırlıklarına göre merkezi sunucuda değerlendirir. Böylece modelin doğruluk oranı daha da yükselir. [27]

## 2.5.1 Federatif Öğrenme Türleri

### 2.5.1.1 Yatay Federatif Öğrenme (Horizontal FL)

Yatay federatif öğrenme veya örnek tabanlı öğrenme, istemci düğümündeki veri kümelerinin aynı özelliklere sahip olduğu ama örneklem uzayının farklı olduğu bir federatif öğrenme yöntemidir. Yani basitçe aynı özelliklere sahip farklı kullanıcıları işlediği bir yöntemdir [28].

Mesela iki hastanenin aynı tür tıbbi testleri uyguladığı bir senaryo düşünelim. Fakat uyguladıkları insan grupları farklı hastalıklara sahipler. Bu durumda kullanılacak en iyi yöntem yatay federatif öğrenmedir.

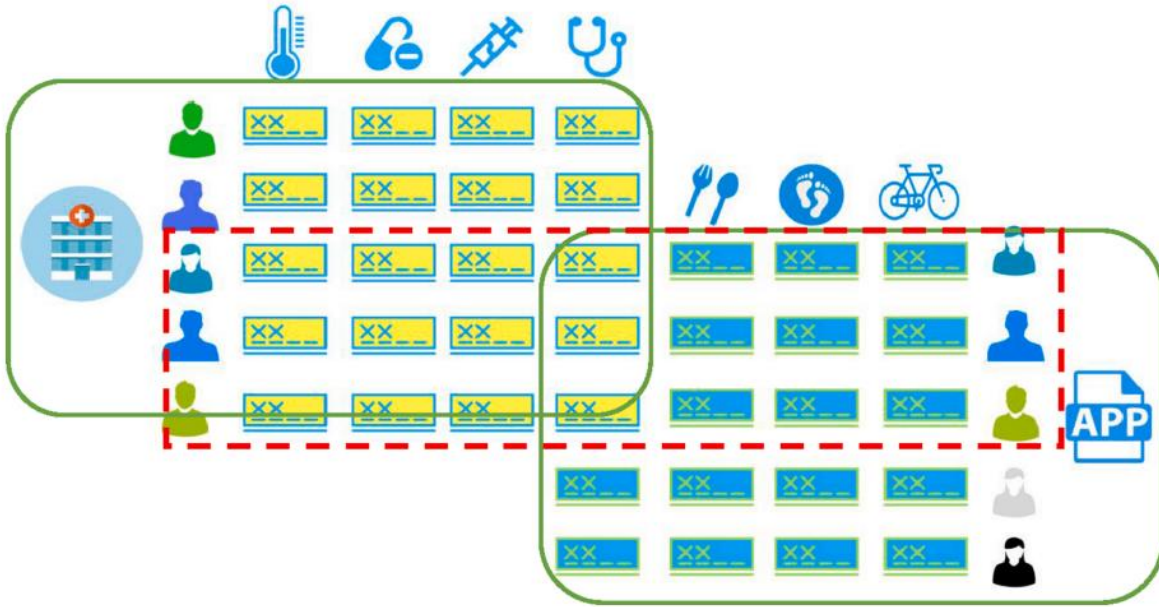


Şekil 2.8 Yatay Federatif Öğrenme

### 2.5.1.2 Dikey Federatif Öğrenme (Vertical FL)

Dikey federatif öğrenme veya özellik tabanlı öğrenme, aynı örneklem uzayında olan ancak farklı özelliklere sahip istemci düğüm veri kümelerini içerir. Yani basitçe farklı özelliklere sahip aynı kullanıcıları işlediği bir yöntemdir [28].

Mesela yine aynı kişinin bir hastaneye kan testi verdiği bir senaryo düşünelim. Farklı bir hastaneye de MR çekti. Bu durumda hasta aynı kişi ama özellikleri farklı. Bu durumda dikey federatif öğrenme kullanmak çok daha mantıklıdır.



Şekil 2.9 Dikey Federatif Öğrenme

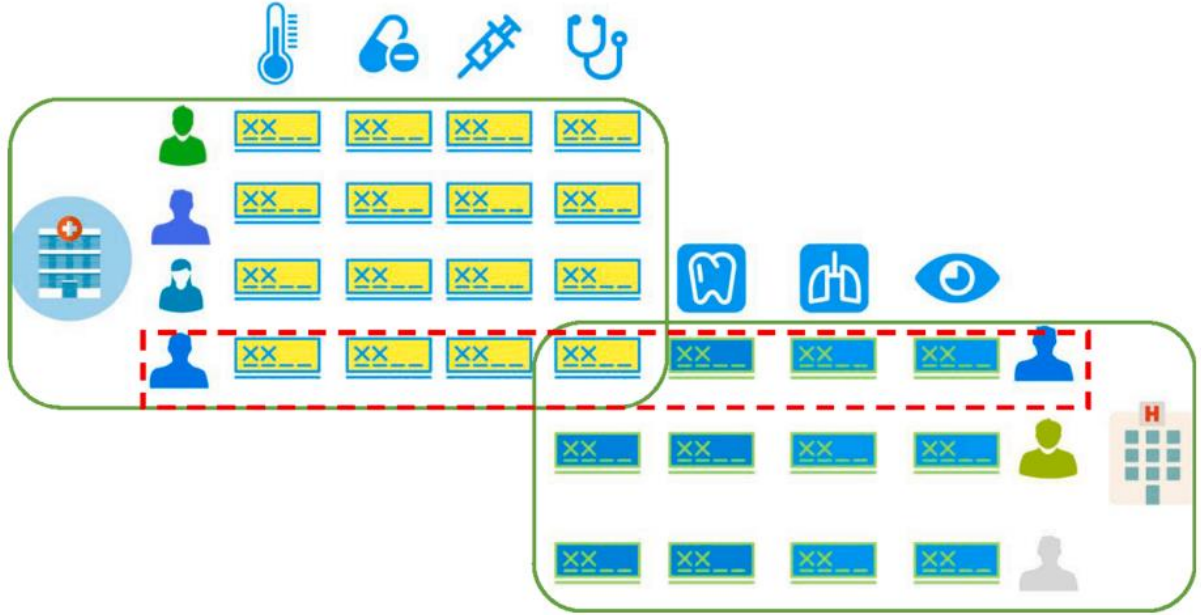
### 2.5.1.3 Karma Öğrenme (Federated Transfer Learning)

Karma öğrenme hem örneklem uzayının hem de kullanıcıların benzerliğinin düşük olduğu noktalarda kullanılan bir federatif öğrenme yöntemidir. Diğer bir ifadeyle, katılımcı kurumların elindeki veri kümeleri farklı özellik alanlarına ve farklı kullanıcı/örnek gruplarına sahiptir ve bu nedenle doğrudan ortak bir model eğitimi yapmak mümkün değildir. FTL yaklaşımı, transfer öğrenme tekniklerinden yararlanarak bu uyumsuzluğu giderir[28].

Bu yöntemde, her bir katılımcının verileri kendisinde kalmaya devam eder; yalnızca öğrenilen özellik temsilleri, model parametreleri veya gradyanlar karşılıklı olarak paylaşılır. Böylece veri gizliliği korunurken, kurumlar birbirlerinin veri yapısından dolaylı biçimde faydalanabilmiş olur.

Mesela farklı hastanelerin farklı bilgilere sahip olduđu bir senaryo düşünelim. Bir hastanede tıbbi görüntü verileri olsun. Diğer hastane ise kan tahlil verilerine sahip.

Bu iki veri türü birbirini tamamlayıcı niteliktedir, ancak aynı hastalara ait olmayabilirler. FTL yöntemi, her iki hastanenin veriyi paylaşmadan daha iyi hastalık risk tahmin modelleri geliştirmesini sağlar.



Şekil 2.10 Karma Öğrenme

### 2.5.2 Federatif Öğrenmeyle Alakalı Yapılan Çalışmalar

Federatif öğrenme, ilk olarak 2016 yılında Google’ın önermesiyle ortaya atılan bir kavramdır. Google, Android kullanıcılar için kullanıcıların yazma ihtimali olan kelimeleri tahmin eden bir klavye uygulaması (GBoard) yaptı. Bu proje, klavyedeki yazılacak kelime tahmininin kalitesini artırırken aynı zamanda kullanıcıların güvenliğini ve gizliliğini de güvence altına aldı.

Ardından birçok araştırmacı bu konular hakkında çalışmalar yapmışlardır. Klavye tahmini konusunda Chen, Mathews, Ouyang ve Beaufays (2019), Leroy, Coucke, Lavril, Gisselbrecht ve Dureau (2019), Hard, Rao, Mathews, Ramaswamy, Beaufays, Augenstein ve Ramage (2019) ve

Yang ve diğeri (2018) tarafından daha fazla iyileştirme yapılmıştır. [69, 70, 71, 72] Ayrıca, emoji tahmini de bir araştırma konusudur. [73] İnsanların davranışlarını tahmin eden birtakım çalışmalar da olmuştur. [74]

Bu olaydan sonra cihazlar eskisinden daha iyi depolama kapasitesi ve hıza sahip olsa da bunun getirdiği dezavantajlardan birisi de bilgi sızıntısının eskiye kıyasla daha yüksek olmasıdır. Buna bir çözüm olarak MEC (Mobil uç bilişim) ve FL (federatif öğrenme) kavramlarını birleştiren Wang ve ark. (2019) kaynak tahsis sorununu da optimize eden 'In-Edge AI' çerçevesini araştırmıştır. [75] Ayrıca Qian ve arkadaşları (2019) da bu konuda araştırmalar yapmışlardır. [76] Sonucunda ise kullanıcılara yakın konumlandırılmış uç sunucular üzerinde ihtiyaç duyulan hizmetleri önbelleğe alarak yüksek kaliteli hizmet sunmayı amaçlayan gizlilik farkındalığına sahip bir hizmet yerleştirme şeması geliştirdiler.

Bu mobil cihazlarda yapılan araştırmalar, IoT cihazlarına da sıçramıştır. Liu, Wang, Liu ve Xu (2020), robot ağlarında robotların taklit öğrenimi için FL tabanlı bir veri birleştirme yaklaşımı önermiştir.

Kullanıcıların tercihlerine göre bir akıllı ev mimarisi oluşturan cihazlar, veri ihlaline sebep olabilir. Bu yüzden Aïvodji, Gambs ve Martin (2019), ortak modeller oluşturan güvenli bir federatif mimarisi ortaya koydular. [77]

Endüstri tarafında da bazı yasal ve yönetmeliklerden dolayı verilere doğrudan erişilebilir olmadığından federatif öğrenme, bu alanda da kullanılabilecek bir yöntemdir.

Bu noktada Hu, Gao, Liu, and Ma (2018) çevre algılama için bölgesel federatif öğrenme (federated-region learning) kullanmışlardır. [78] Böylece çeşitli sensörlerden dağıtılan izleme verileri, global model için kullanılabilir.

Ayrıca federatif öğrenme, görsel incelemede de kullanılmıştır. [79] Üretim görevlerindeki kusurları tespit etmek için hatalı numune eksikliği sorununu çözmeye yardımcı olmakla kalmayıp, aynı zamanda üreticilere gizlilik garantileri de sunmuştur.

Görüntü alanında, görsel ve dilsel bilgiyi birlikte işleyen modeller araştırmaların odak noktalarından biridir. Liu, Wu, Ge, Fan ve Zou (2020), federatif öğrenmeyi kullanarak farklı istemcilerde bulunan görüntü ve dil verilerinden çeşitlendirilmiş temsil öğrenimi elde etmiş ve bu yaklaşımın görsel nesnelerin metinsel ifadelerle daha doğru eşleştirilmesini (grounding) iyileştirdiğini göstermiştir. [80]

Görüntü tespiti dışında federatif öğrenme, İnsansız Hava Araçları (İHA) tarafından oluşturulan kötü amaçlı saldırıların tespiti için de kullanılmıştır. [81] İHA'ların dengesiz veri dağıtımı ve iletişim bağlantılarının güvenilir olması, verilerin merkezde toplanmadan yerel cihazlarda işlenmesini esas alan federatif öğrenme yaklaşımının doğal avantajlarıyla örtüşmektedir.

Yang, Zhang, Ye, Li ve Xu (2019), kredi kartı dolandırıcılığını etkili bir şekilde tespit etmek için farklı bankalara ait işlemlerde federatif öğrenme kullanmıştır. [82] Ayrıca müşterilerin özel bilgilerinin kuruluşlar arasında paylaşılması riskli olduğundan bir risk değerlendirme modeli oluşturmak için federatif öğrenme kullanmak çok mantıklıdır. Cheng, Liu, Chen ve Yang (2020) gizliliği koruyan bir kredi risk değerlendirmesi modeli için federatif öğrenmeyi kullandılar. [83]

Metin madenciliği için Wang, Tong ve Shi (2020), Gizli Dirichlet Tahsisine dayalı endüstriyel düzeyde bir federasyon çerçevesi kullanmaktadır. [84] Bu çerçeve, spam filtreleme ve duygu analizi için gerçek veriler üzerinde yapılan değerlendirmeyi geçmiştir.

Liu, James, Kang, Niyato ve Zhang (2020) gizliliği koruyan bir trafik akış tahmini modeli yapmak için federatif öğrenmeyi kullandılar. [85] Bu model, farklı coğrafi konumlardaki araçları çevrimiçi eğitmek ve özelliklerin doğru şekilde etiketlenmesini kolaylaştırabilmek için tasarlanmıştır.

Federatif öğrenme, NLP alanında da sıklıkla kullanılabilir. Çünkü düzgün bir dil modeli oluşturabilmek için çok fazla veriye ihtiyaç vardır ve bu veriler de insanda olduğu için yine gizlilik konusu göze çarpmaktadır.

Liu, Dligach ve Miller (2019), doktor notları, hikâye öyküleri gibi hastalık tespiti konusunda önemli olabilecek metinleri federatif öğrenme çerçevesinde NLP yöntemini kullanarak işlemiştir. [86] Bu çalışmada iki aşamalı bir federatif eğitim yaklaşımı benimsenmiştir. İlk aşamada, her hasta için klinik metinlerden anlamlı temsiller elde edilmesine yönelik bir ön işleme ve temsil öğrenme süreci yürütülmüş; ikinci aşamada ise bu temsiller kullanılarak çeşitli hastalık fenotiplerinin

sınıflandırılması gerçekleştirilmiştir. Bu çalışma federatif öğrenme ortamında NLP uygulayan ilk çalışma olarak kabul edilmektedir.

Federatif öğrenmeyle alakalı çalışmaların çoğu sağlık sektörüyle alakalı çalışmalardır. Çünkü sağlık sektöründe veri gizliliği diğer alanlara kıyasla daha kritiktir. Bununla alakalı yasal çerçeveler diğer alanlara kıyasla daha nettir.

Çok fazla kategoride elektronik tıbbi kayıtları vardır. Kim, Sun, Yu ve Jiang (2017), hastanın verilerini paylaşmadan sağlık kayıtlarında fenotipleme analizinde tensör çarpanlarına ayırma modellerini kullanmayı denedi. [87] Tıp sektöründe uygulanan ilk federatif öğrenme yöntemi olarak bilinir.

Lee ve arkadaşları (2018), farklı hastanelerde dağınık şekilde bulunan hastalar arasında benzerlik tespiti gerçekleştirmek amacıyla federatif hasta eşleştirme (patient hashing) çerçevesi önermiştir. [88] Bu yöntem, hasta düzeyindeki veriler paylaşılmadan, gizliliği koruyan bir hash mekanizması ile hasta temsillerinin karşılaştırılmasına olanak tanımaktadır. Böylece benzer klinik özelliklere sahip hastaların belirlenmesi mümkün olmakta ve hekimlerin daha önce benzer vakalarda edinilmiş deneyimlerden yararlanarak tedavi stratejilerini geliştirmelerine yardımcı olmaktadır.

Cohen ve arkadaşları (2019), hastaların mahremiyet ihtiyaçlarına göre mevcut yasal ve ahlaki zorlukları analiz etmiş ve gelecekte mahremiyeti ifşa etmeden hasta verilerinden nasıl daha iyi yararlanılabileceğini tartışmıştır. [89]

Silva, Gutman, Romero, Thompson, Altmann ve Lorenzi (2019), farklı hastanelerden gelen manyetik rezonans görüntülerinden hastalık tespiti çıkarmak için federatifle temel bileşen analizi (fPCA) yöntemini önermişlerdir. [90] Ayrıca Gao ve diğerleri (2019), sınırlı etiketli örneklerin yanı sıra gizlilik kısıtlamasının da üstesinden gelmek için Elektroensefalografi (EEG) sınıflandırması için hiyerarşik heterojen yatay federatif öğrenme (HHHFL) çerçevesi önermiştir. [91]

Pfohl, Dai ve Heller (2019), elektronik tıbbi kayıtlar (EMR) üzerinde farklılaştırılmış gizlilik (differential privacy) içeren federatif öğrenme yöntemlerini incelemiştir. [92] Çalışmada, hasta verilerinin gizliliğini korumak amacıyla veriler merkezi bir sunucuda toplanmadan yerel cihazlarda işlenmiştir. Araştırmacılar, gizlilik korumalı federatif öğrenme yaklaşımının

performansının, verilerin merkezi bir yapıda toplandığı geleneksel eğitim yöntemlerine benzer doğrulukta sonuçlar ortaya koyduğunu göstermiştir.

Huang, Shea ve arkadaşları (2019), kalp hastalığına sahip hastaların ölüm oranını tahmin etmek için hastanelerdeki elektronik tıbbi kayıtlardan faydalanmıştır. [93] Eğitim boyunca herhangi bir veri tabanına erişim söz konusu olmamıştır. Merkezi bir sunucuda kullanılacak veriler önceden kodlanmış, eğitim sonunda bu verileri tekrar çözecek yapı tamamen kaldırılmıştır. Ayrıca, Brisimi ve arkadaşları (2018), cPDS (cluster Primal Dual Splitting) adında federatif öğrenmeyi kullanan bir algoritma geliştirerek kalp hastalarının hastaneye yatırılıp yatırılmayacağını değerlendiren bir uygulama yapmışlardır. [94]

Huang, Yin ve çalışma arkadaşları (2019), MIMIC-III veri tabanından elde edilen ilaç kullanım kayıtlarını kullanarak hastaların mortalite riskini tahmin etmek amacıyla kayıp tabanlı uyarlanabilir bir Federated Averaging (Loss-based Adaptive Boosting FedAvg) yaklaşımı geliştirmiştir. [95] Bu yöntem, her istemcinin katkısını kayıp fonksiyonuna göre dinamik biçimde ayarlayarak model eğitiminin etkinliğini artırmaktadır. Çalışmada yalnızca doğruluk değil, istemci başına hesaplama karmaşıklığı ve iletişim maliyetleri de göz önünde bulundurulmuş; böylece önerilen yaklaşım mevcut temel yöntemlere kıyasla daha üstün performans sergilemiştir.

2020’de COVID-19 varyantının çıkmasıyla birlikte federatif öğrenme, bu alanda da çalışmalara konu olmuştur. Dayan ve arkadaşları (2021), laboratuvar verilerine ve röntgen sonuçlarına göre COVID-19 semptomları gösteren hastaların gelecekteki oksijen seviyelerini tahmin eden ve onların sonuçlarını daha iyi tahmin eden bir federatif öğrenme modeli önermiştir. [96]

Ouyang ve arkadaşları (2021), COVID-19 vakalarının erken tespiti için blockchain ve akıllı sözleşme temelli işbirlikçi bir erken uyarı çerçevesi geliştirmiştir. [97] Önerilen modelde, yeni vaka tespitine yönelik uyarı üretme süreci, sağlık kuruluşları, toplumsal kurumlar ve bireyler arasında ortak bir biçimde yürütülmüştür. Bu yaklaşım, verilerin merkezi bir yapıda toplanmasını gerektirmeden çoklu paydaş katılımı ile güvenilir ve şeffaf bir erken uyarı mekanizması sağlamıştır.

Ma ve arkadaşları (2022) federatif öğrenmeye dayalı kanser hastaları için yardımcı bir tanı önermiştir. [98] Federatif modelin yardımıyla doktorlar, hastalar için özel programlar sunabilir ve hastaların yaşam sürecini uzatabilir.

## BÖLÜM 3. YÖNTEM

### 3.1 Bakteri

Bakteriler tek hücreli mikroorganizmalardır. Mikroorganizmalar içerisinde geniş bir yer tutarlar. Genel olarak birkaç mikrometre uzunluğundadırlar. Bakteriler çeşitli şekillere sahiptir. Küresel, spiral, çubuk, virgül şeklinde olabilirler.

Yeryüzünde neredeyse her ortamda mevcuttur. Toprakta, deniz suyunda, okyanus derinliklerinde, asitli sıcak su kaynaklarında radyoaktif atık içerisinde büyüyen türler mevcuttur.

Tipik olarak bir gram toprakta 40 milyon bir mililitre tatlı suda ise bir milyon bakteri yaşamaktadır. İnsan vücudundaki bakteri sayısı insan vücudundaki hücre sayısının 1,3 katıdır. Toplam dünyada 5 nonilyon ( $5 \times 10^{30}$ ) bakteri bulunduğu tahmin edilmektedir. Dünyadaki toplam biyokütlenin çoğunu oluştururlar. Çok sayıda türleri mevcuttur ve günümüzde yeni türler keşfedilmeye devam edilmektedir.

Bazı Bakteri türleri tıp, veterinerlik, biyokimya, gıda endüstrisi, çiftçilik, ilaç endüstrisi gibi alanlarda kritik rol oynamaktadır. İnsan ve çevre açısından sürdürülebilirlik, bağışıklık sistemine destek olma gibi önemli görevlere sahiptir. Bunun yanı sıra bazı türler Solunum yolu enfeksiyonları, verem, kolera, şarbon gibi ciddi hastalıklara sebep olabilmektedir [29,30].

### 3.2 Kullanılacak Olan Veri Seti

Bu projede Polonya Krakow'daki Jagiellonian Üniversitesi Mikrobiyoloji Bölümü tarafından akademik araştırmalar için halka açık olarak oluşturulan DIBas (Digital Image of Bacterial Species) veri seti kullanılmıştır. Bu veri seti her tür için 20 görüntü içeren 33 farklı bakteri türüne ait 660 görüntü içermektedir. Veri setinin içerdiği sınıflar aşağıdaki görselde gösterilmiştir [30].



Şekil 3.1 DIBas veri seti [348, 349]

### 3.3 Veri Ön İşleme

Projede kullanılan orijinal DIBas veri seti .tif uzantılı ve yüksek çözünürlüklü (2048x1532 piksel) görüntülerden oluşmaktadır. Bu yüksek çözünürlüklü verilerin doğrudan derin öğrenme modeline verilmesi performans açısından verimsizdir. Bu nedenle ham veri seti bazı ön işleme adımlarından geçmiştir:

#### 3.3.1 Hiyerarşik Dizin Taraması

Etiket tutarlılığını korumak amacıyla os kütüphanesi kullanılmıştır. Walk metodu ile sınıfları barındıran klasör yapısı rekürsif bir şekilde gezilip işlenmiş yeni verilerde aynı hiyerarşide başka bir klasöre aktarılmıştır.

#### 3.3.2 Format Standarizasyonu

Tıbbi cihazlardan alınan görüntüler gri tonlama (grayscale) gibi farklı renk uzaylarında olabilir. Bakterilerin sınıflandırılmasında gram boyama tekniği önemli bir rol oynamaktadır. Gram boyama bir bakteri örneğinin kimliğinin belirlenmesinde ilk aşama olan gram pozitif ve gram negatif ayrımının yapılmasında kullanılır. Yöntem uygulandıktan sonra mor renkli bakteriler gram pozitif, pembe-kırmızı renkli bakteriler ise gram negatif olarak değerlendirilir. Bu yüzden renk ile ayırt etme özelliğini korumak amacıyla RGB önem arz etmektedir. Ayrıca ResNet mimarisinin giriş katmanları ile uyum sağlamak için tüm görüntüler RGB renk uzayına dönüştürülerek 3 kanallı hale getirilmiş ve .jpg formatına dönüştürülmüştür. [350, 351, 355]

### **3.3.3 Çözünürlük İndirgeme**

Ham görüntüler, veri kaybını minimize eden Lanczos Enterpolasyon yöntemi kullanılarak 256x256 piksel boyutuna indirgenmiştir. Modelin giriş boyutu 224x224 olmasına rağmen görüntüler bilinçli olarak 256x256 boyutuna indirgenmiştir çünkü rastgele kırpma yapabilmek için belli miktarda paya ihtiyaç vardır.

### **3.3.4 Sıkıştırma ve Depolama**

İşlenen görüntüler, disk alanından tasarruf etmek ve veri yükleme hızını artırmak amacıyla JPEG formatında sıkıştırılarak kaydedilmiştir. Bu dönüşüm, 4.2 GB'lık ham veri setinin boyutunu önemli ölçüde düşürerek eğitim süresini hızlandırmaktadır.

### **3.3.5 Hata Yönetimi**

Bozuk veya okunamayan dosyaların eğitim sürecini durdurmasını engellemek amacıyla try-except blokları kullanılmıştır. Algoritma, okunamayan dosyaları atlayarak işlemin sürekliliğini sağlamaktadır.

## **3.4 Eğitim Zamanlı Veri Çoğaltma ve Dönüşümler**

Derin öğrenme modelleri genelleme yeteneklerini artırmak için büyük miktarda veriye ihtiyaç duyar. Kullanılan DIBaS veri setindeki sınıf başına düşen örnek sayısının az olması modelin

eğitimi ezberlemesi (overfitting) riski oluşturur. Bu problemi çözmek amacıyla veriyi disk üzerinde fiziksel olarak çoğaltmak yerine eğitim sırasında bellek üzerinde dinamik olarak çeşitlendiren anlık veri çoğaltma stratejisi uygulanmıştır.

Pytorch kütüphanesi kullanılarak veri çoğaltma için aşağıdaki stokastik (rastgelelik içeren) işlemleri kapsamaktadır:

### 3.4.1 Mekânsal Değişmezliği Engellemek için Geometrik Dönüşümler

Mikroskobik görüntülemelerde bakterilerin petri kabındaki konumu ve yönü türün ayırt edici bir özelliği değildir ve değişkenlik gösterir. Modelin konum ve yöne bağlı olarak öğrenmesini engellemek için bazı dönüşümler uygulanmıştır:

- **Rastgele Kırpma ve Ölçekleme (Random Resized Crop)**

256x256 boyutundaki görüntüden rastgele bir konumda %80-%100 aralığında bir kesit alınarak 224x224 boyutuna yeniden örneklenir. Kesit boyutu 224x224 den büyükse görüntü küçültülür. Kesit boyutu 224x224 den küçükse interpolasyon ile resim genişletilir. Bu işlem modelin bakterinin sadece merkezine değil kenarlarına da odaklanmasını sağlar.

- **Yatay ve Dikey Çevirme (RandomHorizontalFlip-RandomVerticalFlip)**

Görüntüler %50 ihtimalle yatay ve dikey olarak çevrilir

- **Rastgele Döndürme (Random Rotation)**

Bakteri kolonilerinin yönünün bağımsızlığının öğrenilmesi için görüntüler  $\pm 90$  derece aralığında rastgele döndürülür.

### 3.4.2.Fotometrik Dönüşümler

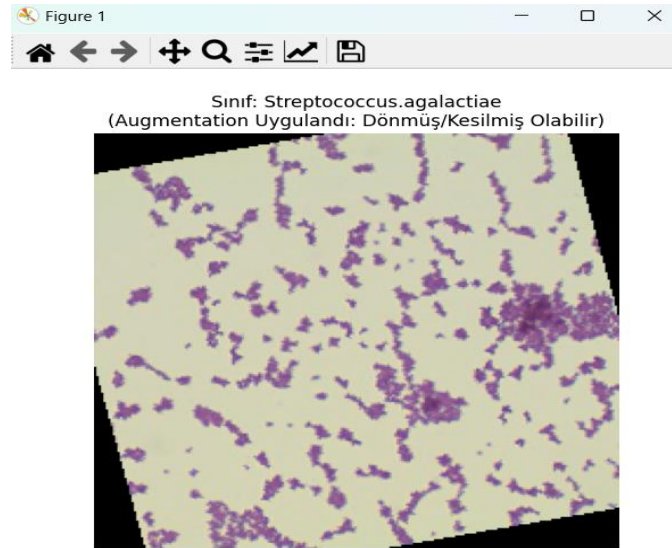
Farklı laboratuvar koşullarındaki mikroskop ışık şiddeti değişimlerini simüle etmek amacıyla, ColorJitter tekniği kullanılmıştır. Görüntülerin parlaklık ve kontrast değerlerinde %10 oranında rastgele sapmalar oluşturularak modelin aydınlatma değişimlerine karşı dayanıklılığı artırılmıştır.

### 3.4.3. Tensör ve Normalizasyon Dönüşümü

Veri çoğaltma işlemlerinin ardından görüntüler [0,255] aralığından [0,1] aralığına ölçeklenerek PyTorch tensörüne dönüştürülmüştür. Daha sonra Transfer Öğrenme mimarisine uyumluluk sağlamak amacıyla tüm veriler ImageNet veri setinin istatistiksel dağılımına RGB kanalları için Ortalama: [0.485,0.456,0.406] Standart Sapma [0.229,0.224,0.225] için değerleriyle normalize edilmiştir.

Normalizasyon:  $\text{Output}[\text{channel}] = (\text{Input}[\text{channel}] - \text{Mean}[\text{channel}]) / \text{Std}[\text{channel}]$

Bu işlem gradyan iniş (gradient descent) optimizasyonu ile hatayı düşürürken daha hızlı ve kararlı yakınsama sağlar. Aşağıdaki görsel dönüşümler uygulandıktan sonra örnek bir görseli temsil etmektedir.



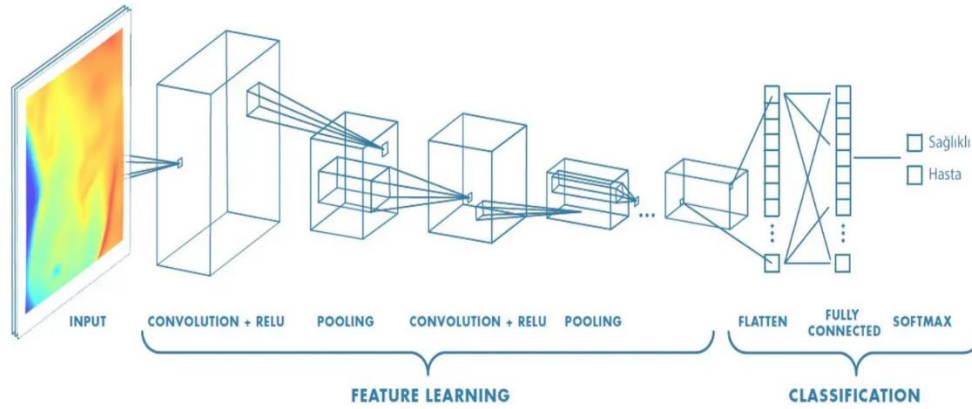
Şekil 3.2 Tensör ve Normalizasyon Dönüşümü

### 3.5 Derin Öğrenme Mimarisi Seçimi

Evrişimli Sinir Ağları (CNN) görüntüler için özelleşmiş bir sinir ağı çeşididir. Bu derin öğrenme ağı metin, görüntü ve ses dahil olmak üzere çeşitli veri türlerine işlemek ve tahmin yapmak için denenmiştir. CNN'ler görüntü işleme için standart yaklaşımdır. [31, 32]

Evrişimli sinir ağlarının temel katmanları aşağıda gösterilmiştir [33]:

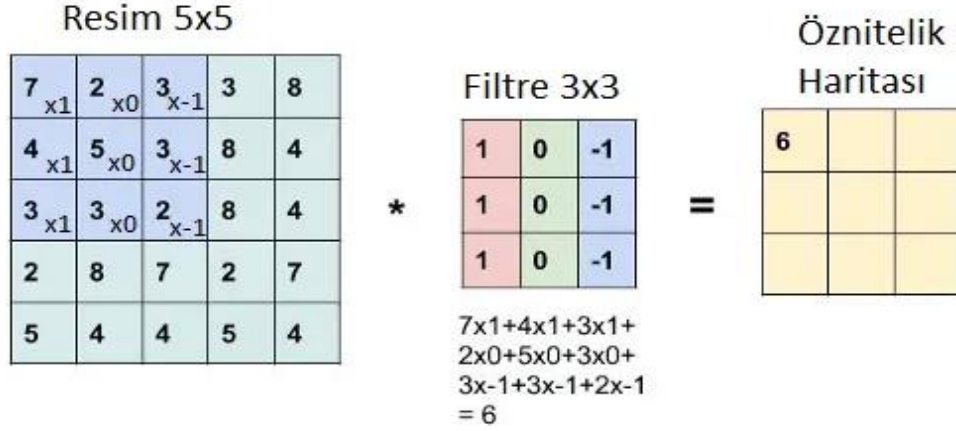
- Konvolüsyon katmanı
- Doğrusal olmayan (Non-linear Activation) katman
- Havuzlama (Pooling) katmanı
- Düzleştirme (Flattening) katmanı
- Tam Bağlı (Fully Connected) Katman



Şekil 3.3 Evrişimli Sinir Ağları

### 3.5.1 Konvolüsyon Katmanı

Bu katman görüntüdeki yüksek ve düşük seviyeli özellikleri çıkarmak için görüntüye filtre işlemi uygulanmasını ifade eder. Belli boyutta bir filtre seçilir ve görüntü ile filtrenin eşleşen indisleri çarpılır. Daha sonra bu çarpımlar toplanır. Filtre görüntü boyunca belirlenen miktarda kaydırılır. Bu kaydırma işlemine stride adı verilir. Satır sonuna gelince alt satıra geçilir. Görüntü üzerinde filtre uygulama işlemi bittikten sonra özellik haritası (feature map) elde edilir. Azalan boyutu telafi etmek amacıyla 0'lar eklenerek boyut artırılabilir. Bu işleme padding adı verilir.



Şekil 3.4 Konvolüsyon Katmanı

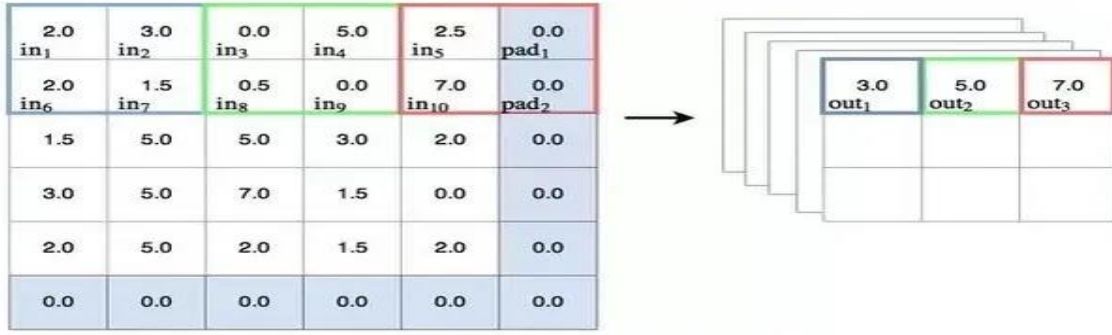
### 3.5.2 Doğrusal Olmayan Katman

Görüntüler doğrusal bir şekilde temsil edilemeyecek kadar karmaşıktır. Konvülisyon katmanı lineer bir süreçtir. Bu sebeple ağ doğrusal olmayan bir katmana ihtiyaç duyar.

Doğrusal olmayan aktivasyon fonksiyonlarına sigmoid, tanh, ReLU örnek verilebilir. ReLU CNN'ler için en yaygın doğrusal olmayan aktivasyon fonksiyonudur. ReLU  $f(x) = \max\{0, x\}$  formülü ile ifade edilir.

### 3.5.3 Havuzlama (Pooling) Katmanı

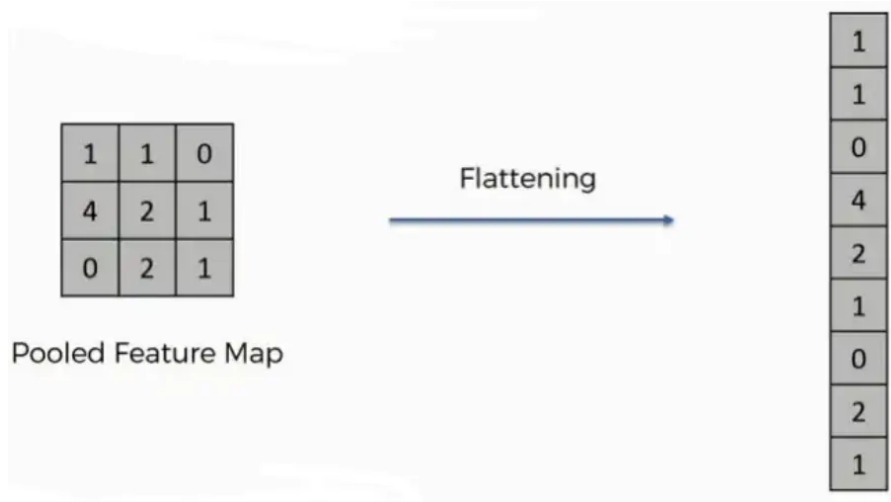
Belli boyutta bir filtre seçilerek özellik haritası üzerinde ilerlenerek boyut indirgeme yapılır. Bu işlem aşırı uyumu engeller. Havuzlama katmanı sonuçların özetlenmesini sağlar ve girdiyi geometrik bozulmalara karşı korur. Max pooling, min pooling ve mean pooling gibi yöntemler mevcuttur. En çok tercih edileni max pooling dir. Max pooling Filtre konumundaki en büyük sayı seçilerek uygulanır. Benzer şekilde min pooling de en küçük, mean pooling de ortalama değer seçilir. Görselde max pooling uygulaması gösterilmiştir.



Şekil 3.5 Havuzlama Katmanı

### 3.5.4 Düzleştirme (Flattening) Katmanı

Konvülsiyon ve Havuzlama katmanından sonra elde edilen matrisin Tam bağlı sinir ağına girişini uyumlu hale getirmek için matris düzleştirilir.



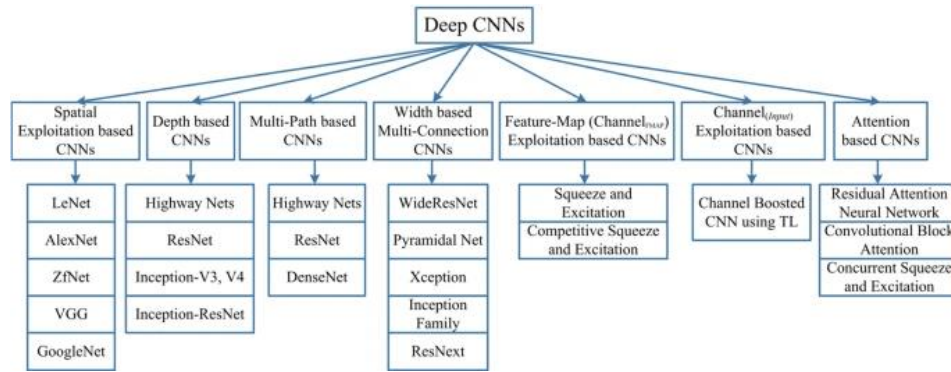
Şekil 3.6 Düzleştirme Katmanı

### 3.5.5 Tam Bağlı (Fully Connected) Katman

Tam bağlı durumu bir katmandaki bir düğümün önceki katmandaki tüm düğümlerle bağlı olduğu durumu ifade eder. Bu katman CNN’de sınıflandırma veya regresyon için kullanılır.

### 3.6 Derin Evrişimli Sinir Ağları

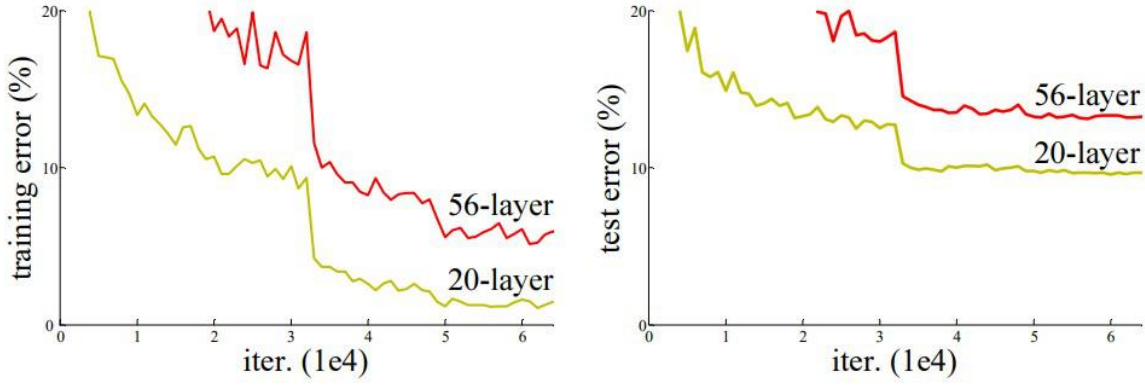
Derin terimi birkaçtan birkaç düzine veya daha fazla evrişim katmanı kullanılan mimarileri ifade eder [34]. Derin mimariler karmaşık öğrenme sorunlarıyla uğraşmada sığ mimarilere göre daha elverişlidir. Birden fazla doğrusal ve doğrusal olmayan katmanların sıralanması karmaşık desenleri öğrenmeyi sağlar. Derin mimariler öğrenme kapasitesini geliştirdiği gözlemlenmiş ve görüntü sınıflandırma da kullanımı artmıştır. Büyük veri kullanımı ve donanımsal gelişmeler de derin mimarilerin başarısında önemli rol oynamaktadır[35].



Şekil 3.7 Derin Evrişimli Sinir Ağları

AlexNet in 2012’de Görsel nesne tanıma araştırmalarında kullanılmak üzere hazırlanan ImageNet veri tabanı üzerinde gösterdiği yüksek başarıdan sonra derin CNN mimarilerine yönelik çalışmalar yaygınlaştı [35]. Daha sonra hata oranını azaltmak için derin sinir ağlarında daha fazla katman kullanılmaya çalışıldı. Ancak bu durum kaybolan gradyan (vanishing gradient) veya gradyan patlaması (exploding gradient) sorununa sebep oldu. Kaybolan gradyan geri yayılım sırasında türev değerlerinin 0 olması veya 0’a çok yaklaşmasını ifade eder. Gradyan Patlaması ise geri yayılım sırasında türev değerlerinin fazla büyük olması ve dengesiz değişimlere sebep olur. Dolayısıyla katman sayısının artması eğitim ve test hata oranının artmasına sebep olur.

Aşağıdaki görselde bu durum 56 katmanlı ve 20 katmanlı CNN karşılaştırılarak gösterilmiştir [36].



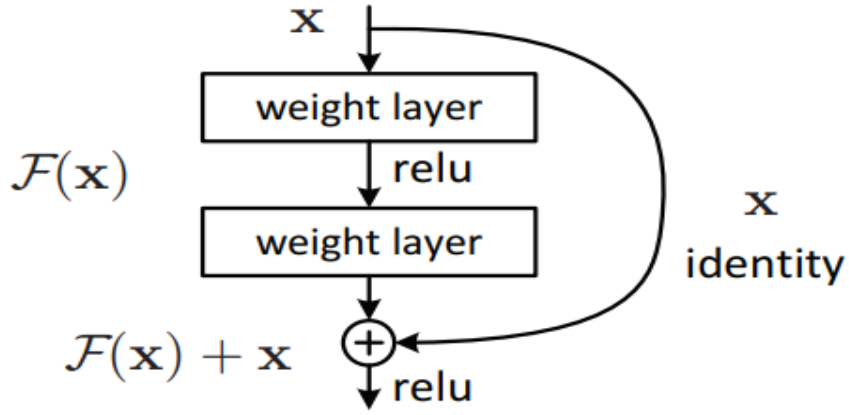
Tablo 3.1 56 Katmanlı CNN – 20 Katmanlı CNN

### 3.7 ResNet

ResNet 2015 yılında Kaiming He, Xiangyu Zhang, Shaoqing Ren ve Jian Sun tarafından önemli ölçüde daha derin olan ağların eğitimini kolaylaştırmak için “Deep Residual Learning for Image Recognition” makalesinde tanıtılan [358] belirli bir sinir ağı türüdür.

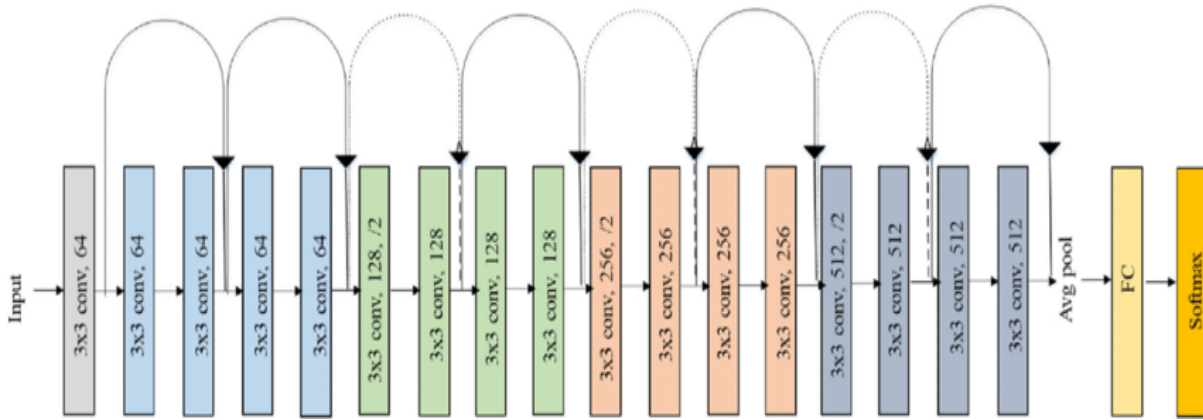
ResNet Artık Bloklar (Residual Blocks) adı verilen bir yapıyı kullanır. Bu yapıda atlama noktaları (skip connections) adı verilen bir teknik kullanılır. Atlama noktaları bir katmanın çıktısını aradaki bazı katmanları atlayarak diğer katmanlara bağlar. Bu bir artık blok oluşturur. ResNet bu artık blokların istiflenmesi ile oluşmuştur. Bu ağın ardındaki yaklaşım Katmanların altta yatan eşlemeyi öğrenmesi yerine, ağın artık eşlemeye uymasını sağlamaktır. Yani,  $H(x)$  öğrenmek istediğimiz fonksiyon ise

$F(x) = H(x) - x$  artıktır. Bunu öğrenmek daha kolaydır. Ağ bu artık  $F(x)$  atlama noktası kullanarak girdi ile birleştirir. Dolayısıyla  $H(x) = F(x) + x$  sonucu elde edilir [36].



Tablo 3.2 ResNet

Atlama noktası kullanmanın amacı eğer herhangi bir katman hatayı artırırorsa  $H(x) = 0$  olarak direkt atlanabilir. Bu sayede kaybolan/patlayan gradyan problemi sorunu olmadan derin ağırlar eğitebilmek mümkün olmaktadır.

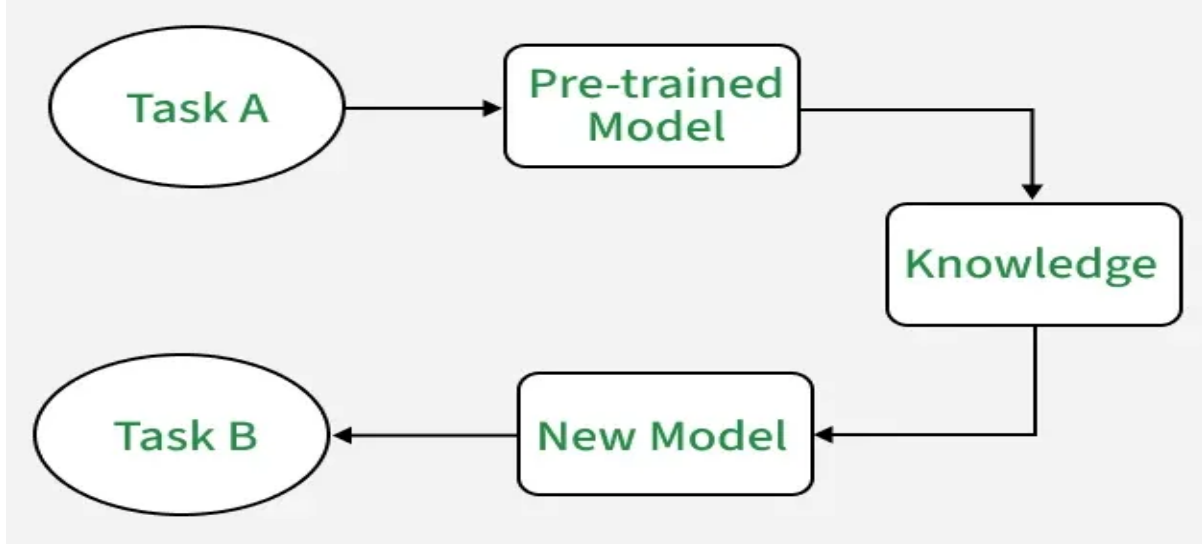


Şekil 3.8 ResNet18 Mimarisi

### 3.8 Transfer Öğrenmesi (Transfer Learning)

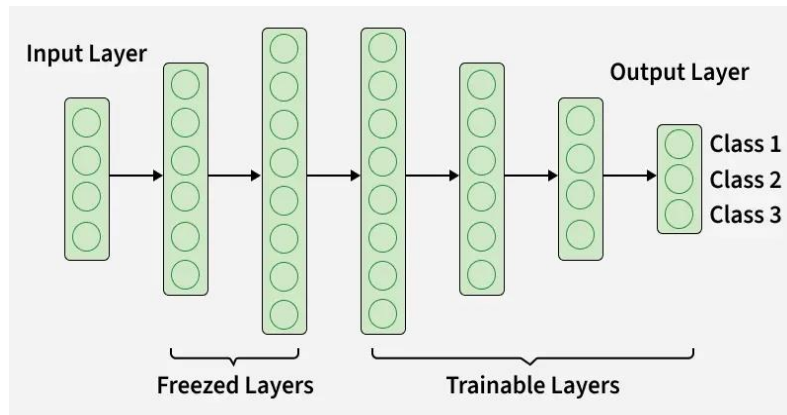
Piyano çalmayı öğrenmek isteyen iki kişiyi düşünelim. Birisi daha önce gitar çalmayı öğrenmiş diğeri ise daha önce herhangi bir müzik aleti çalmadığını varsayarsak gitar çalmayı öğrenen kişinin piyano çalmayı daha kısa sürede öğrenmesi beklenir. Örnekteki gibi Transfer öğrenmesi de bir

görev için eğitilen bir modelin başka bir görev için temel alınarak ince ayar ile yeni göreve uygun hale getirilmesidir. [37]



Tablo 3.3 Transfer Öğrenmesi

İnce ayar modelin temel katmanlarının dondurulması (frozen layers) ve sondaki bazı katmanların (trainable layers) yeni göreve özgü olacak şekilde değiştirilmesidir. Katmanların ne derece dondurulacağı ve ince ayar yapılacağı veri kümesinin benzerliği ve boyutuna bağlı olarak değişir [38].



Şekil 3.9 Transfer Öğrenmesi-2

Sağlık alanında büyük miktarda veri bulmak genellikle zordur. Transfer öğrenmesi önceden eğitilmiş modelleri kullanmamızı sağlayarak büyük veri kümelerine olan ihtiyacı azaltır. Kaliteli veriler ile önceden eğitilmiş bir model ile başlamak yeni görev için eğitim süresini kısaltır ve kaynakları korur. Transfer Öğrenmesinin başarıyla uygulandığı birçok makine öğrenmesi uygulaması vardır. Bunlara metin duygu sınıflandırması, görüntü sınıflandırması, insan etkinliği sınıflandırması, yazılım kusur sınıflandırması ve çok dilli metin sınıflandırması örnek verilebilir. [37]

### **3.9 Federatif Öğrenme Mimarileri**

Geleneksel merkezi makine öğrenimi, çeşitli kaynaklardan veri toplayıp toplanan veriler üzerinde merkezi bir konumda bir makine öğrenimi modeli eğitmeyi içerir. Ancak bu yaklaşımın veri gizliliği endişeleri, pahalı veri aktarım maliyetleri ve sınırlı ölçeklenebilirlik gibi sınırlamaları vardır. Buna karşılık, federatif öğrenme (FL), birden fazla cihaz veya sistemin verileri cihazlarda tutarken ortak bir modeli iş birliği içinde öğrenmesine olanak tanıyan, makine öğrenimine yönelik merkezi olmayan bir yaklaşımdır. Verileri merkezi bir sunucuya göndermek yerine, veriler yerel cihazlarda kalır ve yalnızca model güncellemeleri merkezi bir sunucuya gönderilir [39].

Bu, 2012 tarihli Beyaz Saray tüketici verilerinin gizliliği raporunda önerilen odaklanmış toplama veya veri minimizasyonu ilkesinin doğrudan bir uygulamasıdır. Bu güncellemeler mevcut modeli iyileştirmeye özel olduğundan, uygulandıktan sonra bunları depolamak için hiçbir neden yoktur.

Federasyonlu öğrenmenin başka bir özelliği, veri heterojenliğidir. İstemci verileri merkezden uzak olduğundan, her istemcinin elindeki veri örnekleri bağımsız ve özdeş bir şekilde dağıtılmayabilir [40].

#### **3.9.1 Federatif Öğrenme Adımları**

##### **3.9.1.1 Global Modelin Başlatılması**

Sunucu bir başlangıç modeli kullanarak eğitime başlar. Bu model rastgele parametreler ile başlatılabilir veya önceden eğitilmiş bir model kullanılabilir. Sistemdeki tüm cihazların ortak olarak kullanacağı ilk modeldir [41].

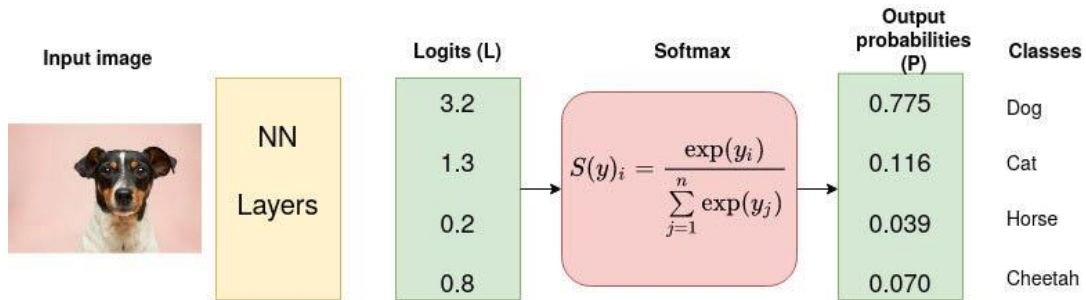
### 3.9.1.2 İstemcilere Modelin Dağıtılması

İlk model istemcilere gönderilir. İstemciler için başlangıç modelidir [41].

### 3.9.1.3 Yerel Eğitim Döngüsü

İstemciler ilk modeli kendi yerel verileriyle eğitir. Eğitim sürecinde belirli bir batch size seçilir. Batch size her eğitim adımında (iterasyon) modele aynı anda verilecek örnek sayısını belirtir. Bir iterasyon tek bir batch in işlenmesi anlamına gelir. Tüm veri setinin model tarafından bir kez görülmesi ise epoch olarak adlandırılır. Model, batch'i işledikten sonra bir kayıp fonksiyonu (loss function) aracılığıyla tahmin ile gerçek değer arasındaki hatayı hesaplar. Kayıp fonksiyonları problem tipine göre değişir: regresyon problemlerinde genellikle ortalama kare hata (mean squared error, MSE) veya ortalama mutlak hata (mean absolute error, MAE) kullanılırken, sınıflandırma problemlerinde binary cross entropy (ikili sınıflandırma) ve categorical cross entropy (çok sınıflı sınıflandırma) tercih edilir.

Categorical cross entropy şöyle çalışır: Öncelikle Modelin son katmanı logit adı verilen ham skorlar üretir. Softmax fonksiyonu bu logit değerlerini 0 ile 1 arasına sıkıştırarak toplamaları 1 olacak şekilde olasılık dağılımına çevirir.



Tablo 3.4 Federatif Öğrenme-2

Daha sonra categorical cross entropy formülünün çalışması için gerçek değerın one hot encode edilmesi gerekir. One hot encoding doğru kategoriye 1 diğer kategorilere 0 değeri verilerek ikili formatta vektörize etme işlemidir. [42]

| Color |  | Red | Blue | Green |
|-------|--|-----|------|-------|
| Red   |  | 1   | 0    | 0     |
| Blue  |  | 0   | 1    | 0     |
| Green |  | 0   | 0    | 1     |

Tablo 3.5 Federatif Öğrenme-3

Bu adımdan sonra categorical cross entropy işlemi uygulanabilir.

$$L(y, \hat{y}) = - \sum_{i=1}^c y_i \log(\hat{y}_i)$$

$L(y, \hat{y})$ : Kayıp değeri

$C$ : Toplam sınıf sayısı

$y_i$ :  $i$ 'inci sınıf için gerçek etiket (one-hot)

$\hat{y}_i$ :  $i$ 'inci sınıf için modelin tahmini olasılığı

Kayıp, gerçek sınıfın karşılık geldiği olasılığın logaritmasının alınması ve one-hot vektörü ile çarpılmasıyla hesaplanır. Bu işlem, yanlış sınıflara verilen yüksek olasılıkları daha fazla cezalandırır [43].

Hesaplanan kaybı minimize etmek için modelin ağırlıkları, geri yayılım (backpropagation) algoritması ile güncellenir. Geri yayılım sırasında gradyanlar zincir kuralı kullanılarak hesaplanır ve optimizasyon algoritması aracılığıyla ağırlıklara uygulanır

#### 3.9.1.4 Güncellemelerin Sunucuya Geri Gönderilmesi

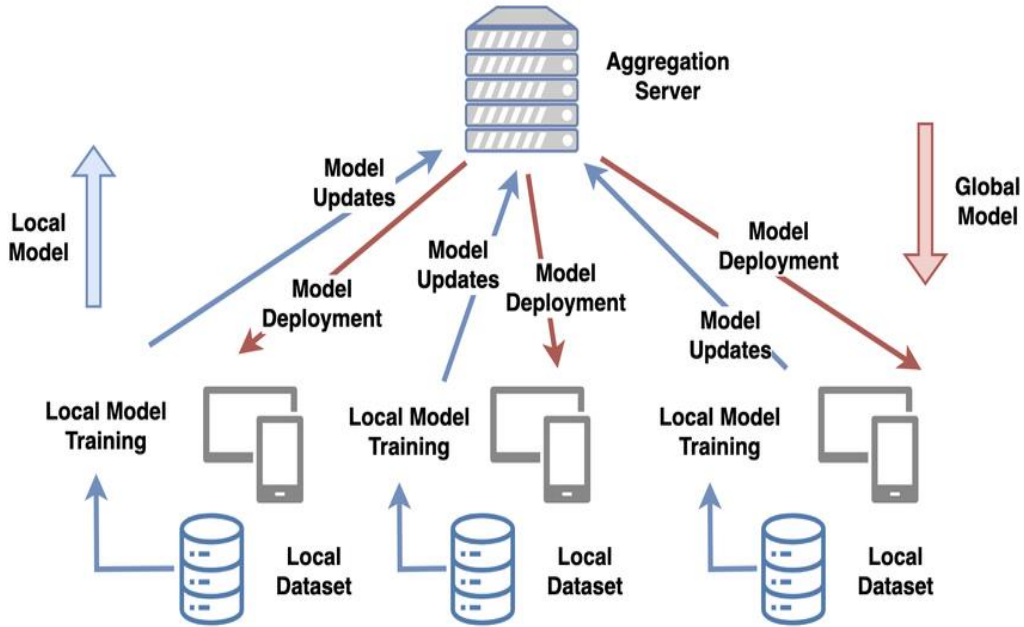
Sistem içindeki tüm istemcilerde eğitim tamamlandıktan sonra her bir istemci sahip olduğu tüm güncel model parametrelerini Sunucuya gönderir [41].

### 3.9.1.5 Sunucuda Global Modelin Güncellenmesi

Sunucu tüm istemcilerden aldığı güncel parametrelerini FedAvg, FedProx gibi algoritmaları kullanarak tek bir global modelde birleştirir. [41]

### 3.9.1.6 Güncel Global Modelin Yeniden Cihazlara Dağıtımı

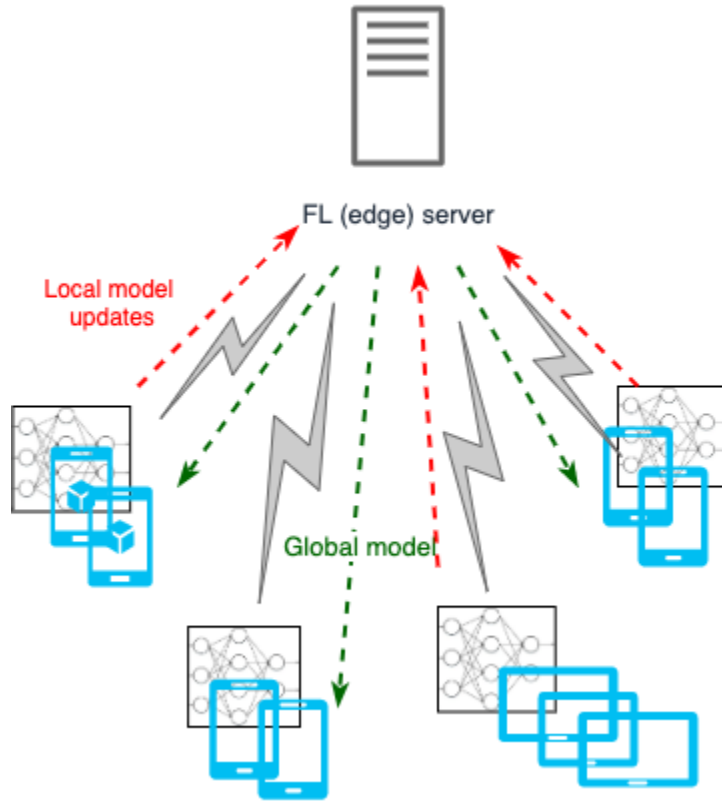
Cihazlardan gelen parametreler ile oluşturulan yeni global model istemcilere gönderilir. Bu adımlar iteratif bir şekilde devam eder. Aşağıdaki görsel federatif öğrenme sürecini göstermektedir. [41]



Şekil 3.10 Güncel Global Modelin Yeniden Cihazlara Dağıtımı

### 3.9.2 Merkezi Federatif Öğrenme (CFL)

Merkezi bir sunucu, her bir modelin nasıl bir araya getirilebileceğini kontrol eder. Her bir istemci, ağırlıklarını ve modellerinin ayrıntılarını bu merkezi sunucuya gönderir. Merkezi sunucu, bu model ayrıntılarını ve ağırlıkları günceller ve daha gelişmiş, iyileştirilmiş modeli istemcilere geri gönderir. İstemciler bu ana modeli alır, kendi verilerini kullanarak eğitir ve güncellenmiş modeli geri gönderir. Bu döngü daha iyisi olana kadar devam eder. Bu yöntem, her ne kadar kontrol etmesi kolay olsa da merkezi sunucuya herhangi bir şey olursa sistem çöker [44].

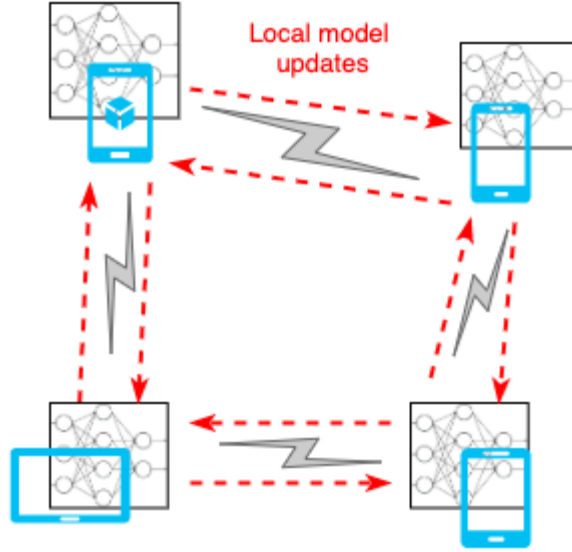


Şekil 3.11 Centralized Federated Learning (DFL)

### 3.9.3 Merkezi Olmayan Federatif Öğrenme (DFL)

Merkezi bir sunucudan ziyade her bir istemcinin birbirleriyle beraber çalışma prensibine dayanarak çalışan DFL, istemcilerin kendi ağırlıklarını, bağlı olduğu istemcilere atarak küresel bir model oluşturmayı amaçlar. Bu eşler arası (peer-to-peer) yaklaşımı, tek bir hata noktasını ortadan

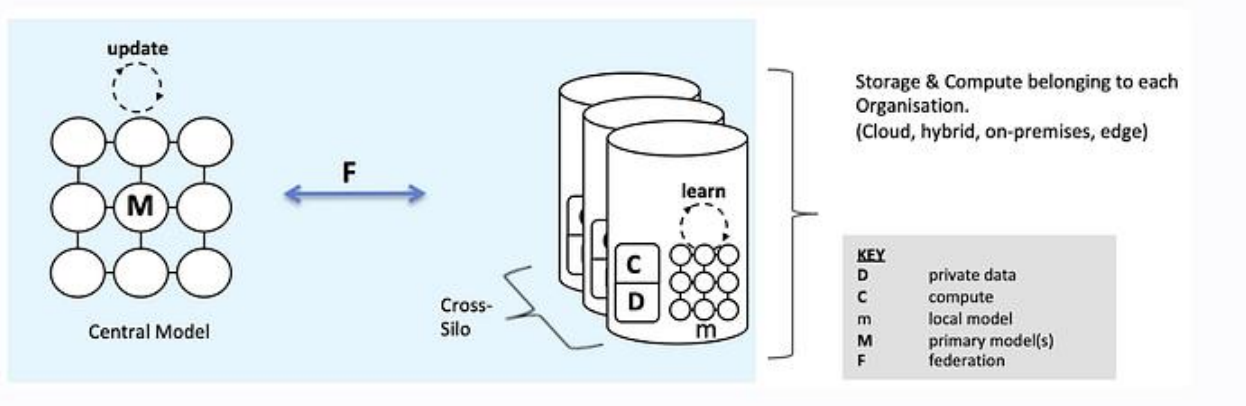
kaldırarak sistemi daha güçlü yapar. Bu yöntem gizliliği ve dayanıklılığı artırırken sistemi yönetmek daha da zorlaşabilir [44].



Şekil 3.12 Decentralized Federated Learning (CFL)

### 3.9.4 Cross-Silo Federatif Öğrenme

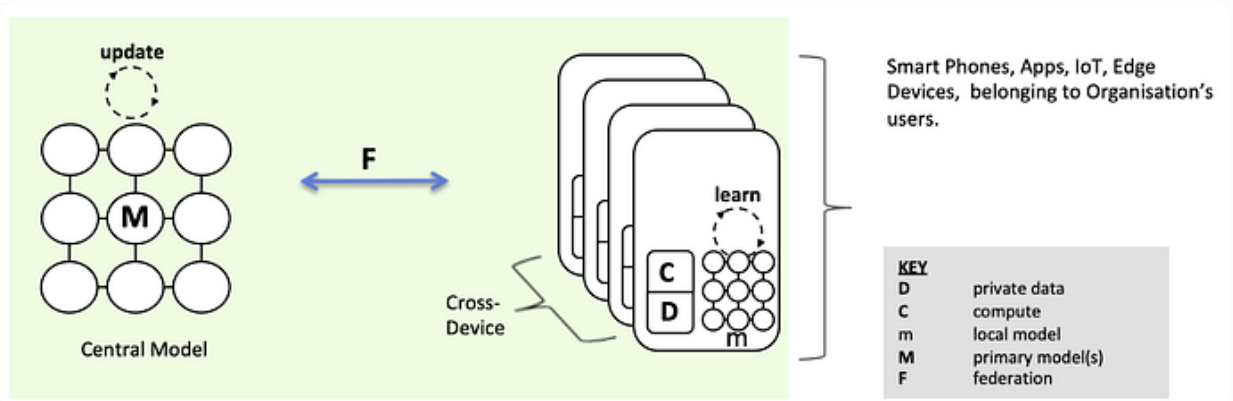
Birbirinden bağımsız fakat kurumsal ölçekte faaliyet gösterenler (ör. hastaneler, bankalar, şirketler veya araştırma kurumları) verilerini paylaşmadan ortak bir makine öğrenimi modeli eğitmek için iş birliği yapar. Bu yaklaşım, veri gizliliği, yasal kısıtlamalar ve rekabetçi kurumlar arasındaki güven eksikliği nedeniyle verinin merkezi bir noktada toplanmasının mümkün olmadığı senaryolarda tercih edilir [44].



Şekil 3.13 Cross-silo Federated Learning

### 3.9.5 Cross-Device Federatif Öğrenme

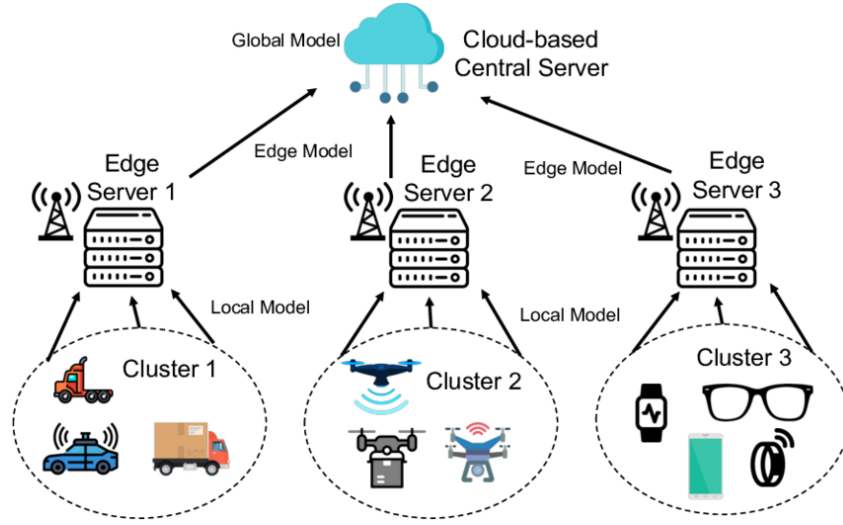
Bu yöntem, akıllı telefonlar, IoT cihazları, bilgisayarlar gibi küçük çaplı internete sahip birçok cihazı kullanır. Bu cihazlar, yerel verilerini paylaşmadan ortak bir makine öğrenimi modeli eğitmek için bir araya gelir. Bu yaklaşım, küresel ölçekte dağıtık cihazlardan gelen gerçek zamanlı ve kullanıcıya özgü verilerin gizlilikten ödün vermeden kullanılabilmesini amaçlar. Bununla birlikte çok sayıda kullanıcıyı ve çok çeşitli verileri de yönetir. Örneğin, Google Klavye, Android cihazlarda metin tahmini ve otomatik düzeltme özelliklerini iyileştirir [44].



Şekil 3.14 Cross-device Federated Learning

### 3.9.6 Hiyerarşik Federatif Öğrenme

Federatif öğrenme sürecinin tek bir merkezi sunucu ile uç cihazlar arasında gerçekleşmek yerine, birden fazla katmanda organize edildiği çok seviyeli bir dağıtık öğrenme mimarisidir. Bu yapı, büyük ölçekli ve heterojen ağlarda iletişim verimliliğini artırmak, gecikmeyi azaltmak ve daha esnek bir FL mimarisi sunmak için geliştirilmiştir [44].



Şekil 3.15 Hierarchical Federated Learning (FL) architecture. [https://www.researchgate.net/figure/Hierarchical-FL-following-tree-topologies-Typically-the-FL-network-has-a-tree-structure\\_fig5\\_368305322](https://www.researchgate.net/figure/Hierarchical-FL-following-tree-topologies-Typically-the-FL-network-has-a-tree-structure_fig5_368305322) Federated Learning

### 3.9.7 Hibrid Federatif Öğrenme

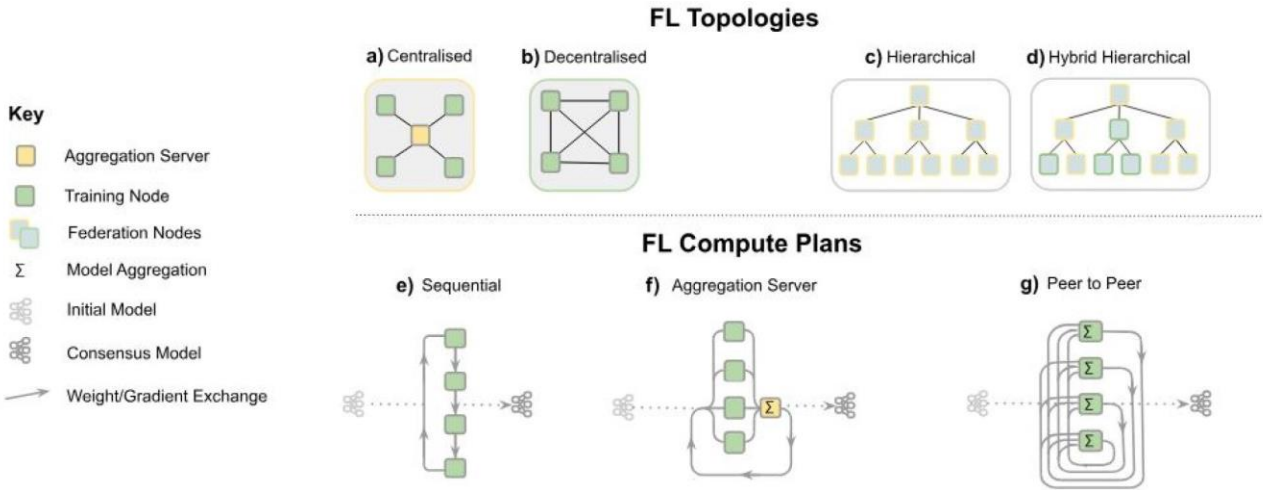
Federatif öğrenmenin hem hiyerarşik hem de heterojen cihaz-kurumsal (cross-device + cross-silo) yapıları birleştirilerek yürütüldüğü hibrit bir federatif öğrenme mimarisidir. Bu yaklaşımda eğitim süreci yalnızca çok katmanlı değildir; aynı zamanda farklı türde katılımcılar aynı öğrenme sürecinin parçası olur [44].

### 3.9.8 Takviyeli Federatif Öğrenme

FL ve RL (reinforcement learning) yöntemlerini birleştirir. Farklı cihazların RL deneyimlerinden elde ettikleri politika güncellemelerini gizli tutarak federatif şekilde birleştiren, çoklu ajan RL ve federatif öğrenmenin birleşiminden oluşan bir yöntemdir [44].

### 3.9.9 Güvenli Federatif Öğrenme

Federatif öğrenme sürecinde veri gizliliğini, iletişim güvenliğini ve model bütünlüğünü sağlamak için kriptografik ve istatistiksel yöntemleri birleştirir [44].



Şekil 3.16 Tüm Federatif Öğrenme Topolojileri ve Hesaplama Yöntemleri

### 3.9.10 Projede kullanılacak olan mimari: Cross-silo

Bu federatif öğrenme mimarilerinden bu proje kapsamında en mantıklı seçenek cross-silo mimarisidir. Öncelikle yapılan projeyi biraz daha açalım.

Laboratuvar ortamındaki bakterilerin sınıflandırmasını sağlayan bu projede, federatif öğrenme kullanılmasının en büyük sebebi verilerin herhangi bir laboratuvar tarafından görülememesi ve gizlilik yasalarını çiğnememesidir.

Laboratuvarlar verilerinin başka yerlere çıkamaması ve görece kurumsal olması, cross-silo mimarisinin benimsediği “kurumsal arası ortak model geliştirme” ilkesine tam anlamıyla uyur.

### 3.10 Federatif Öğrenmede Kullanılan Yöntemler ve Algoritmalar

#### 3.10.1 Federated Stochastic Gradient Descent (FedSGD)

Federatif öğrenmede global modelin merkezi sunucu tarafından, her cihazın gönderdiği gradyanlarla güncellendiği en temel algoritmadır.

- İlk başta merkezi sunucu global modeli  $w_0$  değeri ile başlatır. Her cihaz ( $i$ ) kendi veri kümesi ( $D_i$ ) üzerinde **bir batch** kullanarak gradyan hesaplar.

$$g_i = \nabla F_i(w)$$

- Hesapladığı bu gradyanı sunucuya yollar. Sunucu bu gönderilen tüm gradyanları ortalara ve sonra global modeli günceller.

$$g = \sum_{i=1}^K \frac{n_i}{n} g_i w_{t+1} = w_t - \eta g$$

- Güncellenen model, cihazlara gönderilir ve süreç tekrar edilir.

#### 3.10.2 Federated Averaging (FedAvg)

Verilen cihazda kalmasını, sadece ağırlıklarının paylaşarak ortak bir model oluşturmaya çalışan bir algoritmadır.

- İlk başta merkezi sunucu global modeli  $w_0$  değeri ile başlatır. Her cihaz ( $i$ ) kendi veri kümesi ( $D_i$ ) üzerinde birkaç local SGD adımı yapar.

$$w_i^{t+1} = w_i^t - \eta \nabla f_i(w_i^t)$$

- Bu eğitilen modellerin parametreleri sunucuya tüm yerel modelleri ağırlıklı olarak ortalara.

$$w_{t+1} = \sum_{i=1}^K \frac{n_i}{n} w_i^{t+1}$$

$n_i$  = cihazın veri sayısı

$n = \sum_i n_i$  = toplam veri

$K$  = katılımcı cihaz sayısı

- Sunucu en sonunda global modeli günceller ve bu güncellenen modeli cihazlara geri yollar. Bu birkaç epoch daha devam eder.

Bu algoritmanın en büyük dezavantajı, çok farklı veri modellerinde dengesiz ve kararsız öğrenmesidir. Bu durum, farklı koşullarda federatif ortalama algoritmasını kullanmayı zorlaştırmaktadır. Bunun için bunun üstüne inşa edilen bir algoritma yöntemi ortaya konmuştur.

### 3.10.3 Federated Proximal (FedProx)

FedProx (Federated Proximal Optimization), FedAvg'in bir varyantıdır ve cihazlar arasındaki veri dağılımı farklılıklarını, hesaplama gücü farklılıklarını, eğitim süresi eşitsizliklerini daha iyi yönetmek için ekstra bir proximal term ekler. [46] Yani yerel modellerin, global modelden çok sapmasını engeller. Bunu engellemek için FedAvg algoritmasından farklı olarak yerel loss fonksiyonuna bir ceza termi (proximal term) ekler.

$$F_i(w) + \frac{\mu}{2} \|w - w^{(t)}\|^2$$

$w$  = yerel model

$w^{(t)}$  = global model

$\mu \rightarrow$  ceza katsayısı

Bu algoritma, aynı zamanda eksik veya yarım kalan cihazlardan gelemeyecek olan veriyi de sorun etmez. Çünkü bunu ceza katsayısı ile tolare eder.

### 3.10.4 Federated Dynamics (FedDyn)

Bu algoritma da yine FedAvg algoritmasının eksiklerini kapatmak için kullanılır. İstemciler her eğitim turunda bir “dinamik regularization (dynamic correction)” terimi kullanır ve böylece yerel modellerin global modelden uzaklaşması engellenir. Bu dinamik regularization terimi, geçmiş turdan kalan bir dinamik düzeltme terimidir.

$$\min_w F_i(w) + \langle h_i, w \rangle + 2\alpha \left\| w - w^{(t)} \right\|^2$$

$F_i(w) \rightarrow$  client'in kayıp fonksiyonu

$h_i \rightarrow$  dinamik düzeltme terimi

$\alpha \rightarrow$  regularization gücü

$w^{(t)} \rightarrow$  global model

Yine algoritma adımları FedAvg algoritmasından farklı olarak bu formül üzerinden şekillenir. Veri farklılığı (non-IID) toleransı, FedProx'a göre daha gelişmiştir. Daha hızlıdır ve stabilitesi de daha yüksektir. [47]

### 3.10.5 Adaptive Federated Optimization Algorithms (Federatif Uyarlanabilir Optimizasyon Algoritmaları)

Diğer algoritmalarından farklı olarak bu algoritmalar, sunucu tarafında uyum sağlayabilen bir bileşenle birleştirilip model güncellemesi yaparlar.

FedAdam, sunucu tarafında momentum (m) ve ikinci moment (v) tutar. Her roundda istemcilerin gradyan benzeri güncellemelerini toplar ve sunucu birtakım güncellemeler yapar. Bu FedAvg algoritmasından daha hızlı öğrenmesini sağlar.

FedAvagrad, sunucu tarafında her parametre için kare toplamı tutar ve bunları güncellemelere dahil eder. Bu da çok güncellenen parametrelerin learning rate'inin otomatik olarak düşmesini sağlar.

FedYogi, Adam'a benzemekle birlikte ikinci momenti daha yavaş büyütür. Bu da kararsız verilerde stabilityi artırır. [48]

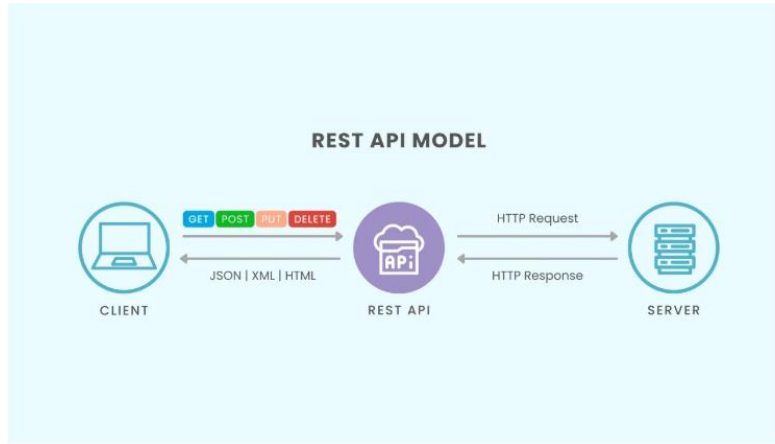
### 3.10.6 Proje İçin Kullanılacak Olan Algoritmalar

Proje kapsamında kullanılması düşünülen iki algoritma var: FedAvg ve FedProx. FedAvg, çoğu zaman cross-silo mimarisinde yeterli olan bir algoritma. Fakat bazen görüntü dağılımları çok farklı (Non-IID) olabilir. Bu durumda FedProx devreye girer ve bu durumu örtbas eder.

### 3.11 Federatif Öğrenmede Kullanılan İletişim Protokolleri

#### 3.11.1 Rest API

Genel olarak web sitelerinde de kullanılan Rest API, federatif öğrenmede de kullanılan yaygın bir protokoldür. Kısaca istemci ile sunucu arasındaki haberleşmeyi sağlayan HTTP protokolü üzerinden çalışan bir mimaridir. Özellikle kurumsal ağlarda kolay yönetilebilen bir protokol olması sebebiyle cross-silo için oldukça uygun bir protokoldür. [49]



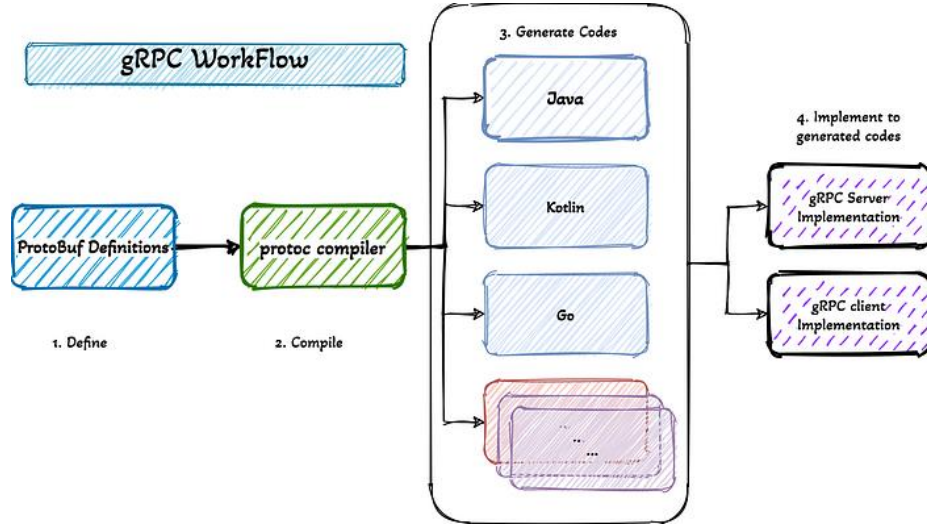
Şekil 3.17 Rest API akış şeması

#### 3.11.2 gRPC

Google tarafında geliştirilen ve ücretsiz bir framework olan RPC protokolü, hızlı ve iletişimi kolay olmasıyla öne çıkar. Açılımı Remote Procedure Call (Uzaktan Adım Çağrısı) olan bu protokol, uzakta olan bir sunucudaki metodu sanki istemcinin kendi metoduymuş gibi kullanılmasını sağlar.

gRPC protokol buffer (ProtoBuf) kavramı, verileri ikili sayılara dönüştürerek taşınabilir bir formata getirir. Giden veri, yine gRPC'nin yardımıyla okunabilir bir formata dönüşür. Sürekli bağlantı, streaming, bi-directional gibi iletişimleri sağlar.

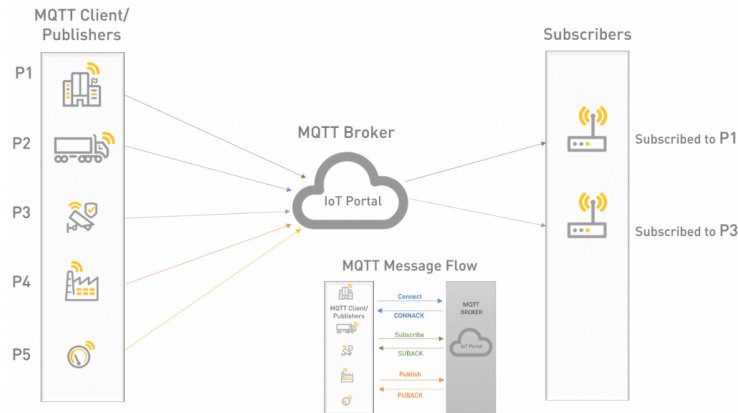
Bu noktada hızlı olması, parametre transferi ve metadata için çok önemlidir. Bu yönden bakarsak cross-silo için uygun olduğunu söyleyebiliriz. [50]



Şekil 3.18 gRPC'nin akış şeması

### 3.11.3 MQTT

Genelde IoT cihazlarında kullanılan bu protokol, publish/subscribe modeliyle çalışır. MQTT'de iki temel bileşen vardır. Broket ve istemci. Broker, aslında bir sunucudur. İstemciler ise istek gönderen aygıtlardır. İstemcinin broker'a veri göndermek istemesi yayımlama (publish), Broker'ın istemciye veri göndermesi abone (subscribe) olarak tanımlanır. Daha çok cross-device mimarisinde kullanılır. [51]

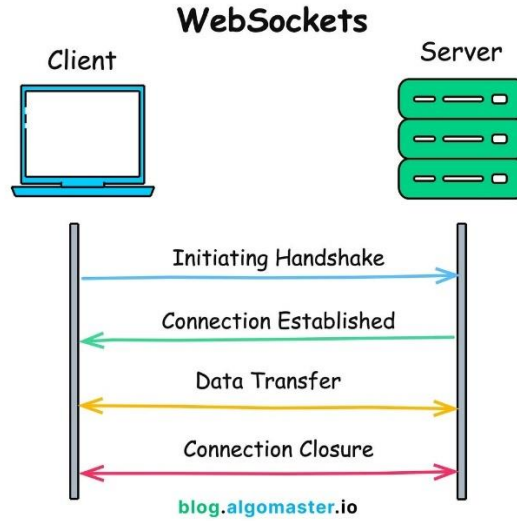


Şekil 3.19 MQTT'nin akış şeması [114]

### 3.11.4 WebSocket

Gerçek zamanlı çift yönlü iletişim sağlayan WebSocket, özellikle çok fazla istemci-sunucu arasında veri transferi olduğu zamanlarda kullanılır. WebSocket, bağlantıyı HTTP ile başlatır. İstemci-sunucu arasındaki bağlantı, taraflardan herhangi birinin bağlantıyı kesmek istemesine kadar sürekli olarak devam eder. Bu da gerçek zamanlı bir web uygulaması geliştirilmesine yardımcı olur.

Yapısı itibariyle hem cross-device hem de cross-silo için kullanılabilir ama kurumsal taraflarda daha çok HTTP ve gRPC kullanılır.



Şekil 3.20: WebSocket'in akış şeması

### 3.11.5 Proje İçin Kullanılacak Olan Protokol: gRPC

Proje kapsamında kullanılması planlanan protokol gRPC'dir. Bu protokolü seçme nedenimiz, protokolün Rest'e kıyasla daha hızlı olmasıdır. Modellerin ağırlıkları paylaşılırken buna ihtiyacımız olabilir. Diğer yandan gRPC, özellikle metadataların dağıtımları konusunda çok iyidir. Bu yüzden projede önemli bir yer taşıyabilir.

### 3.12 Federatif Öğrenmede Gizlilik ve Güvenlik Katmanları

Federatif öğrenmede gizliliğin ne kadar önemli olduğuna değinmiştik. Verilerin hiçbirisi sunucuya aktarılmaz, bu verilerin ağırlık parametreleri aktarılır. Fakat bu durum, birtakım saldırganların bundan faydalanmasına ve verilerin çalınmasına sebep olabilir.

#### 3.12.1 Saldırı Yöntemleri

##### 3.12.1.1 Membership Inference Attacks (Üyelik Çıkarım Saldırısı)

Bu saldırının amacı, modelin parametrelerinden bir örneğin eğitim setinde olup olmadığını anlamaktır. [58] Örneğin saldırgan, gizli bir hastanın MR görüntüsünün eğitim modeline dahil olup olmadığını bu saldırı yöntemiyle bulmaya çalışır. Bu proje kapsamında örnek vermek gerekirse *Bacteroides Fragilis* bakteri türünün bu eğitim modelinde olup olmadığını öğrenmek istemesi diyebiliriz.

##### 3.12.1.2 Property Inference Attacks (Özellik Çıkarım Saldırısı)

Saldırgan, verilerin içindeki istatistiksel özellikleri bulmaya çalışır. Bu proje kapsamında örnek vermek gerekirse bakteri türlerinin görsellerinde yüzde kaçının belirli bir koşul içerdiğini, bakteri türlerinin model eğitilirken veri setlerindeki yüzdesi veya onların meta bilgilerini tahmin edebilir. Verinin kendisine ulaşamaz ama gizli özellikleri bulabilir.

##### 3.12.1.3 Model Poisoning Attacks (Model Zehirleme)

Saldırgan, modellerin bütününde manipülasyon yapar ve modelin yanlış öğrenmesini sağlamaya çalışır. [58] Verileri çalmaz ama modellerin davranışlarını değiştirir. Backdoor koyarak modelin belirli bir düzendeki verilerin yanlış sınıflandırmasını sağlar. Bu proje kapsamında örnek vermek gerekirse *Escherichia coli* bakterisini backdoor koyarak farklı bir bakteri sınıflandırmasına sokarak modelin davranışlarını değiştirebilir.

#### 3.12.1.4 Data Poisoning Attacks (Veri Zehirleme)

Saldırgan, modellerin eğitim setinde kendi verisini ekleyerek modelin işleyişini bozar. Etiket çarpıtarak modelin yanlış öğrenmesini sağlar. Bu şekilde modeli bozar ve modele olan güveni zayıflatır. Bu proje kapsamında örnek vermek gerekirse *Lactobacillus jensenii* bakterisiyle ilgili eğitim veri setine alakasız veriler koyar ve bu da modelin bu bakteriyle alakalı sınıflandırmasını bozar.

#### 3.12.1.5 Gradient Inversion Attack (Derivatif Tersine Çevirme Saldırısı)

Saldırgan, model güncellemeleri üzerinden orijinal veriyi oluşturmaya çalışır. [58] Sunucuya giden gradyanları alır. Gradyanı minimize eden bir giriş görüntüsünü optimize eder. Böylece görüntü yüksek kaliteyle elde edilebilir. Bu saldırı, özellikle verinin kendisini almayı amaçlar.

### 3.12.2 Korunma Yöntemleri

#### 3.12.2.1 Differential Privacy (DP)

Gradyanlara veya çıktılara gürültü ekler. [58] Böylece saldırgan, neyin ne olduğunu anlayamaz. Özellikle ilk maddedeki *üyelik çıkarım* ve *gradient inversion saldırısına* engel olur. Fakat parametrelere gürültü eklemek, modelin doğruluğunu bozabilir.

#### 3.12.2.2 Secure Aggregation (Güvenli Birleştirme)

Sunucuyu kontrol eden kişi, istemcilerden gelen gradyanlarının her birini ayrı ayrı göremez. Sadece toplam veya ortalama gradyanları görebilir. Böylece *gradient inversion saldırısı* tamamen etkisiz hale gelir.

### 3.12.2.3 Homomorphic Encryption (Tam Homomorfik Şifreleme)

Bu korunma yönteminde veri, daha istemcideyken şifrelenir ve sunucu, şifreli gradyanlar üzerinden çalışır. [58] Böylece sunucu gradyanı açamaz ve veri hakkında hiçbir bilgi alamaz. Ama çok büyük bir dezavantajı var. O da bu yöntemin çok yavaş olması.

### 3.12.2.4 Trusted Execution Environments (SGX / TEE)

Model güncellemeleri, güvenli bir ortam sağlayan TEE platformunda işlenir. Böylece sunucu bile içerideki verilere dokunamaz. TEE ortamı, yalnızca CPU aygıtları için uygundur.

### 3.12.3 Proje Kapsamında Kullanılacak Olan Korunma Yöntemleri

Proje kapsamında kullanılması planlanan yöntemler; *Secure Aggregation (Güvenli Birleştirme)* ve *Differential Privacy (DP)*. Bir yandan verilerin ve gradyanların korunması önemli ancak tam *homomorfik şifreleme* gibi yöntemler sistemi çok fazla yavaşlatabilir. Öte yandan TEE gibi tam koruma sağlayan yöntemler kullanılabilir ancak maliyet açısından sıkıntılı. Bu yüzden bu proje kapsamına dahil edilmemiştir. Kimlik doğrulama (authentication) ile istemci veya sunucu tarafına erişim kısıtlanmış olur.

## BÖLÜM 4. UYGULAMA

### 4.1 Uygulama ve Deney Ortamı

Yüksek hacimli veriler, model eğitim aşamasında RAM üzerinde tutulup işleme tabi tutulacağından, mevcut donanım kapasitesi yetersiz kalabilir ve daha güçlü bir sisteme ihtiyaç duyulabilir. Ayrıca, bu tür büyük verilerle eğitilen modeller yüksek işlem gücü gerektirdiğinden, donanım yetersiz kaldığında eğitim süreci inanılmaz derecede uzun sürebilir ve daha güçlü bir GPU veya TPU'ya ihtiyaç duyulabilir. Özellikle çalışılan veri türü görsel formatta ise modelin veriyi işleyebilmesi için GPU kullanımı kritik hale gelir [53].

#### 4.1.1 Jupyter Notebook

Jupyter Notebook, veri analizi ortamı sağlayan açık kaynaklı bir araçtır ve genellikle Python, R gibi veri analizi yapmayı sağlayan programlama dillerini destekler. Ayrıca Jupyter; makine öğrenimi, kodlama ve veri analizi gibi çeşitli uygulamalar için ideal olan, etkileşimli ve zengin bir belge biçimidir [54].

#### 4.1.2 Google Colab

Google Colaboratory, yaygın adıyla Google Colab, 2017 yılında Google Research tarafından geliştirilen ve Python tabanlı veri bilimi ile makine öğrenimi projelerini çalıştırmaya olanak tanıyan ücretsiz bir bulut tabanlı platformdur.

Makine öğrenimi gibi yoğun hesaplama işlemleri için ücretsiz GPU ve TPU desteği sağlamaktadır.

Google Drive ile entegre çalışan Colab, kullanıcıların Jupyter Notebook ortamında kod yazmasına, çalıştırmasına ve paylaşmasına imkân tanır. Drive bağlantısı sayesinde ilgili verileri depolayarak Colab içerisinde veriler hızlı bir şekilde model eğitimi için kullanılabilir. Jupyter Notebook ortamında yazılan kod sayesinde eğitilen modellerin sonuçları bir yerde depolanmak için .ipynb formatında indirilerek çalıştırılan kod bloklarının sonuçlarını da tutarak bir depolama işlemi gerçekleştirilebilir [53].

#### 4.1.3 Kullanılan Kütüphaneler

Uygulamanın geliştirilmesinde, yapay zekâ ve veri bilimi ekosisteminde kabul görmüş, yüksek performanslı ve açık kaynaklı kütüphaneler kullanılmıştır.

- **Python (3.10+):** Sistemin ana programlama dili.
- **PyTorch (2.x):** Model mimarisinin kurulması, gradyan hesaplamaları ve federatif ağırlık birleştirme (FedAvg) algoritmasının uygulanması için kullanılan temel çerçeve.
- **Torchvision:** Önceden eğitilmiş (Pretrained) modellerin ve görüntü transformasyon (augmentations) işlemlerinin yönetimi.
- **Scikit-learn:** Karmaşıklık matrisi (Confusion Matrix) ve başarımların metriklerinin hesaplanması.
- **Seaborn & Matplotlib:** Eğitim sonuçlarının görselleştirilmesi ve yüksek çözünürlüklü ısı haritalarının (Heatmaps) oluşturulması.

#### 4.2. Deneysel Veri Yapılandırması ve Gizlilik Simülasyonu

Bu bölüm, tasarlanan federatif sistemin veriyi nasıl yönettiğini, gizlilik prensiplerini nasıl simüle ettiğini ve modelin başarısını ölçmek için kullanılan veri stratejisini detaylandırmaktadır.

##### 4.2.1 Veri Bölümleme Stratejisi

Modelin genelleme yeteneğini (hiç görmediği verilere karşı başarısını) ölçmek amacıyla, ön işleme aşamasından geçmiş olan bakteriyel görüntüler iki ana gruba ayrılmıştır:

- **Eğitim Seti:** Toplam veri setinin %85 i federatif döngü içerisinde istemciler tarafından kullanılmak üzere ayrılmıştır.
- **Global Doğrulama Seti:** Toplam veri setinin %15'lik kısmından oluşturulan doğrulama seti, federatif eğitim döngüsü boyunca görülmemiş veri olarak tutulmaktadır. Her tura (round) ait ağırlık birleştirme işlemi bittikten sonra güncel global model, bu veri seti üzerinde test edilir. Bu strateji, modelin yerel veri setlerine aşırı uyum sağlamasını engelleyerek global bir genelleme başarısı elde edilip edilmediğini denetler.

#### 4.2.2 İstemci Dağılımı ve Veri İzolasyonu

Federatif öğrenme mimarisini simüle etmek için 3 bağımsız sanal istemci (Hastaneler) oluşturulmuştur. Eğitim seti, her bir istemciye rastgele ve dengeli bir şekilde paylaştırılmıştır.

- **Yerel Veri Bağımsızlığı:** Her istemci sadece kendine ayrılan Subset (alt küme) verisini görür. Bir istemcinin verisi, eğitim sırasında asla başka bir istemciye veya merkezi sunucuya aktarılmaz.
- **Rastgele Dağıtım:** torch.randperm fonksiyonu kullanılarak veriler karıştırılmış, böylece her hastanenin farklı bakteri sınıflarına sahip olması sağlanarak simülasyonun gerçekçiliği artırılmıştır.

#### 4.2.3 Gizlilik Protokolü ve Simülasyonu

Sistemde veri gizliliği, görüntülerin kendisi yerine sadece model ağırlıklarının transfer edilmesiyle simüle edilmektedir.

- **Ağırlık Transferi:** İstemciler yerel eğitimlerini tamamladıktan sonra sadece state\_dict (katman ağırlıkları) bilgisini sunucuya gönderir.

- **Veri Sızıntısı Engelleme:** Sunucu, istemcilerin içindeki ham piksellere erişemez. Bu durum, sağlık verilerinin gizliliğini koruyan dijital bir simülasyondur.

#### 4.2.4 Veri Artırma ve Doğrulama Stratejisi

Sistemde eğitim ve doğrulama aşamaları için birbirinden farklı veri işleme stratejileri izlenmiştir. Bu farklılık, modelin sadece görüntüleri ezberlemesini önlemek ve gerçek dünya değişkenlerine karşı dayanıklılığını artırmak amacıyla kurgulanmıştır.

- **Eğitim Aşaması:** İstemcilerdeki eğitim verilerine rastgele döndürme, yatay ve dikey çevirme, rasgele kırpma ve ölçekleme renk değişimleri ve perspektif kaydırma gibi veri artırma teknikleri uygulanarak modelin zor koşullarda öğrenmesi sağlanmıştır.
- **Doğrulama Aşaması:** Global doğrulama setindeki görüntülerde hiçbir yapay değişim yapılmamıştır. Bu aşamada görüntüler sadece modelin giriş boyutu olan  $224 \times 224$  ölçüsüne normalize edilerek sabitlenmiştir.

Eğitim sırasında uygulanan veri artırma teknikleri, veriyi tahmin edilebilmesi için daha zor bir hale getirir. Bu durum eğitim hatasını artırır. Bu veri artırma teknikleri doğrulama setine uygulanmadığı için doğrulama hatası eğitim hatasından daha düşük olabilir [55].

#### 4.3 Model Mimarisi ve Parametre Seçimi

Bu bölümde, bakteriyel kolonilerin sınıflandırılması için tercih edilen derin öğrenme mimarisi, bu mimarinin federatif öğrenme sistemine adaptasyonu ve eğitimin kararlılığını sağlayan parametre seçimi detaylandırılmaktadır.

##### 4.3.1 ResNet18 Mimarisi ve Transfer Learning

Projede temel öznetelik çıkarıcı (feature extractor) olarak ResNet18 mimarisi tercih edilmiştir. ResNet mimarisinin seçilme nedenleri şunlardır:

- **Artık Bağlantılar (Skip Connections):** Derin ağlarda görülen “kaybolan gradyan” (vanishing gradient) problemini, katmanlar arası doğrudan bağlantılarla çözerek eğitimin daha derin katmanlarda bile başarılı olmasını sağlar.
- **Transfer Learning Avantajı:** Model, devasa bir veri seti olan ImageNet üzerinde önceden eğitilmiş (pretrained) ağırlıklarla yüklenmiştir. Bu sayede model; kenar, doku ve renk gibi temel görsel özellikleri zaten bilmekte, sadece bakteri morfolojisine odaklanmaktadır.

#### 4.3.2. Model Modifikasyonu ve Özelleştirme

ResNet18 mimarisi orijinalinde 1000 farklı nesneyi tanıyacak şekilde tasarlanmıştır [56]. Projedeki 33 farklı bakteri türüne uyum sağlaması için model üzerinde şu modifikasyon yapılmıştır:

- **Tam Bağlantılı Katman (FC Layer):** Modelin son katmanı olan fc (fully connected) katmanı 512 girişli ve 33 çıkışlı bir doğrusal katmanla değiştirilmiştir.
- **Çıkış Aktivasyonu:** Model, her bir sınıf için bir skor üretmekte ve bu skorlar *CrossEntropyLoss* fonksiyonu içerisinde işlenerek olasılık değerlerine dönüştürülmektedir.

#### 4.3.3. FedAvg (Federated Averaging) Algoritması

Sistemde istemcilerden gelen bilgilerin birleştirilmesi için **FedAvg** algoritması kullanılmıştır. Bu algoritma, yerel verileri paylaşmadan merkezi bir model oluşturulmasını sağlar. Her turun sonunda sunucu, belli sayıdaki istemciden gelen yerel ağırlıkları toplar ve bunların ortalamasını alarak yeni global modeli oluşturur. Bu yöntem, veri gizliliğini ihlal etmeden her hastanenin tecrübesini tek bir modelde birleştirmeyi sağlar.

#### 4.3.4 Hiper-Parametre Konfigürasyonu

| Parametre                            | Değer |                                                                                                |
|--------------------------------------|-------|------------------------------------------------------------------------------------------------|
| <b>Toplam Tur (NUM_ROUNDS)</b>       | 180   | Merkez sunucu ile istemciler arasındaki toplam iletişim ve bilgi birleştirme sayısıdır.        |
| <b>Öğrenme Oranı (Learning Rate)</b> | 0.001 | Modelin her adımda hata payını düzeltirken attığı adımın büyüklüğünü belirler.                 |
| <b>Momentum</b>                      | 0.9   | Öğrenme sürecine hız (ivme) kazandırarak hatalı yollara sapılmasını ve duraksamaları önler.    |
| <b>Optimizasyon Algoritması</b>      | SGD   | Modelin hatalarını en aza indirmek için kullandığı temel matematiksel güncelleme yöntemidir.   |
| <b>(Batch Size)</b>                  | 32    | Tek bir işlem adımında modelin aynı anda incelediği görüntü miktarını ifade eder.              |
| <b>Yerel Epoch</b>                   | 1     | Her istemcinin, ana merkeze bilgi göndermeden önce kendi verisiyle yapacağı tam tur sayısıdır. |

Tablo 4.1 Hiper-Parametre Konfigürasyonu

#### 4.4 Deneysel Bulgular ve Başarım Analizi

Bu bölümde, geliştirilen federatif öğrenme modelinin 180 turluk simülasyon sürecinden elde edilen sayısal veriler, öğrenme eğrileri ve modelin sınıflandırma yeteneği detaylı olarak analiz edilmektedir.

#### 4.4.1 Eğitim Sonuçları

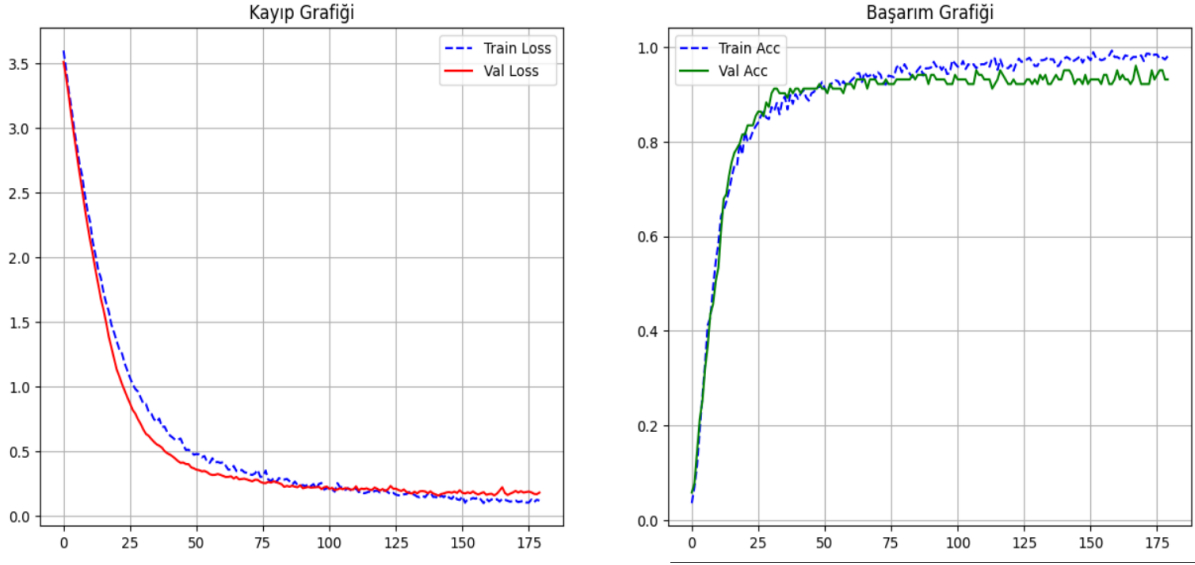
| Round | Train Acc     | Val Acc       | Train Loss    | Val Loss      |
|-------|---------------|---------------|---------------|---------------|
| 1     | 0.0358        | 0.0583        | 3.5988        | 3.5103        |
| 20    | 0.7730        | 0.8155        | 1.4177        | 1.2123        |
| 40    | 0.9010        | 0.9126        | 0.6471        | 0.4835        |
| 60    | 0.9232        | 0.9223        | 0.4087        | 0.3163        |
| 80    | 0.9471        | 0.9320        | 0.2900        | 0.2654        |
| 100   | 0.9659        | 0.9417        | 0.2120        | 0.2254        |
| 120   | 0.9556        | 0.9223        | 0.1804        | 0.1948        |
| 140   | 0.9710        | 0.9320        | 0.1459        | 0.1721        |
| 160   | 0.9795        | 0.9320        | 0.1322        | 0.1622        |
| 168   | <b>0.9795</b> | <b>0.9612</b> | <b>0.1159</b> | <b>0.1602</b> |
| 180   | 0.9812        | 0.9320        | 0.1174        | 0.1790        |

Tablo 4.2 Eğitim Sonuçları

Eğitim süreci boyunca modelin ulaştığı tepe performans noktasını muhafaza etmek adına her turda doğrulama başarısı (best\_val\_acc) anlık olarak takip edilmiştir. Simülasyon sırasında rekor bir başarıya ulaşıldığı her an, modelin mevcut ağırlıkları otomatik olarak best\_bacteria\_model.pth dosyasına kaydedilerek fiziksel bir veri kaydı oluşturulmuştur. Bu mekanizma, eğitimin son aşamalarında ortaya çıkabilecek olası performans dalgalanmalarına veya aşırı öğrenme belirtilerine karşı bir "güvenlik sigortası" görevi görerek, sistemin yakaladığı en

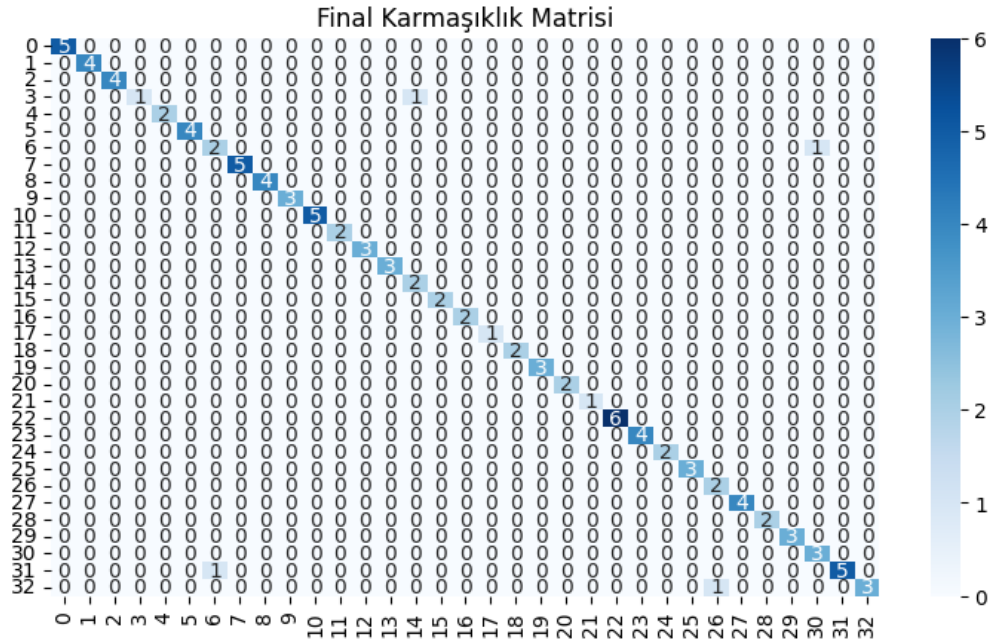
yüksek doğruluk oranının kayıpsız bir şekilde saklanması ve nihai testlerde kullanılmasını sağlamıştır.

#### 4.4.2 Eğitim ve Doğrulama Sürecine Ait Kayıp ve Başarım Grafikleri



Tablo 4.3 Eğitim ve Doğrulama Sürecine Ait Kayıp ve Başarım Grafikleri

#### 4.4.3 Karmaşıklık Matrisi



Tablo 4.4 Karmaşıklık Matrisi (Alt taraftaki numaralar, her bir bakterini türünü temsil etmektedir)

## BÖLÜM 5. SONUÇLAR VE ÖNERİLER

Bu tasarım çalışması neticesinde, sağlık sektöründeki veri mahremiyeti kısıtlamalarına yenilikçi bir çözüm olarak geliştirilen Cross-silo mimarili federatif öğrenme sisteminin, mikroskobik bakteri sınıflandırma görevinde uygulanabilirliği ve başarısı somut verilerle ortaya konmuştur. DIBaS veri seti üzerinde ResNet18 mimarisi ve FedAvg algoritması kullanılarak gerçekleştirilen 180 turluk simülasyon süreci sonunda, %97,95 oranında eğitim başarımı ve %96,12 oranında doğrulama başarımı elde edilerek, verilerin merkezi bir sunucuda toplanmadan da yüksek doğruluklu modellerin eğitilebileceği kanıtlanmıştır.

İleride gerçekleştirilecek çalışmalar ve sistemin geliştirilmesi adına, veri dağılımının heterojen ve dengesiz olduğu (Non-IID) senaryolarda model kararlılığını korumak için FedAvg yerine FedProx veya FedDyn gibi hata toleransı yüksek algoritmaların sisteme entegre edilmesi önerilmektedir. Bununla birlikte, veri güvenliğini en üst seviyeye taşımak amacıyla, ileride gerçekleştirilecek çalışmalarda Secure Aggregation ve Differential Privacy kullanılması, sistemin siber saldırılara ve veri sızıntılarına karşı dayanıklılığını önemli ölçüde artıracaktır.

Ayrıca bu projede, yazılım geliştirme sürecinin daha düzenli ve takip edilebilir olması amacıyla **Git versiyon kontrol sistemi** kullanılmıştır. Git sayesinde projede yapılan değişiklikler kayıt altına alınmış, hatalı durumlarda önceki sürümlere geri dönebilmiş ve kod geliştirme süreci adım adım izlenebilmiştir. Projenin uzak depo (remote repository) olarak **GitHub** platformu kullanılmıştır. Tüm kaynak kodlar GitHub üzerinden paylaşılmıştır. [1]

## KAYNAKLAR

- [1] [https://www.beyaz.net/tr/yazilim/makaleler/yapay\\_zeka\\_nedir.html](https://www.beyaz.net/tr/yazilim/makaleler/yapay_zeka_nedir.html)
- [2] <https://www.ibm.com/think/topics/machine-learning>
- [3] <https://medium.com/machine-learning-türkiye/makine-ogrenmesi-7cfbb3d859db>
- [4] <https://nyilmazsimsek.medium.com/derin-öğrenme-deep-learning-nedir-ve-nasıl-çalışır-2d7f5850782>
- [5] [https://www.beyaz.net/tr/yazilim/makaleler/derin\\_ogrenme\\_deep\\_learning\\_nedir.html](https://www.beyaz.net/tr/yazilim/makaleler/derin_ogrenme_deep_learning_nedir.html)
- [6] [https://acikerisim.subu.edu.tr/yayinaea/tam-metin\\_67e26b0ff0616.pdf](https://acikerisim.subu.edu.tr/yayinaea/tam-metin_67e26b0ff0616.pdf)
- [7] <https://www.globenewswire.com/news-release/2021/10/27/2321714/0/en/Global-Artificial-Intelligence-AI-Market-to-Reach-US-291-5-Billion-by-the-Year-2026.html>
- [8] 15. Mesko B. Tıbbın Geleceğine Yolculuk. İstanbul: Optimist Yayın Grubu, 2018; 174-183.
- [9] [https://aacrjournals.org/cancerres/article/77/4\\_Supplement/S6-07/624543/Abstract-S6-07-Double-blinded-validation-study-to](https://aacrjournals.org/cancerres/article/77/4_Supplement/S6-07/624543/Abstract-S6-07-Double-blinded-validation-study-to)
- [10] <https://hbr.org/2018/05/10-promising-ai-applications-in-health-care>
- [11] Mergen M, Junga A, Risse B, Valkov D, Graf N, Marschall B. Immersive training of clinical decision making with AI driven virtual patients – a new VR platform called medical tr.AI.ning. GMS Journal for Medical Education. 2023;40.
- [12] Rathod K. Research on Revolutionising Learning with AI Powered Chatbots and NLP Techniques. International journal of Scientific Research in Engineering & Management. 2024.
- [13] <https://www.scmp.com/news/china/science/article/2163298/doctors-said-coma-patients-would-never-wake-ai-said-they-would>
- [14] <https://www.wired.com/story/cancer-risk-ai-mammograms>
- [15] <https://www.siemens-healthineers.com/perspectives/mso-digital-twin-mater.html>
- [16] <https://dergipark.org.tr/en/download/article-file/3065337>

[17] <https://www.proquest.com/docview/3143978849?pq-origsite=gscholar&fromopenview=true&sourcetype=Dissertations%20&%20Theses>

[18] Yadav, Neel; Pandey, Saumya; Gupta, Amit; Dudani, Pankhuri<sup>1</sup>; Gupta, Somesh<sup>1</sup>; Rangarajan, Krithika. Data Privacy in Healthcare: In the Era of Artificial Intelligence. *Indian Dermatology Online Journal* 14(6):p 788-792, Nov–Dec 2023. | DOI: 10.4103/idoj.idoj\_543\_23

[19] M. Ali, F. Naeem, M. Tariq and G. Kaddoum, "Federated Learning for Privacy Preservation in Smart Healthcare Systems: A Comprehensive Survey," in *IEEE Journal of Biomedical and Health Informatics*, vol. 27, no. 2, pp. 778-789, Feb. 2023, doi: 10.1109/JBHI.2022.3181823.

[20] <https://medium.com/@naveenjain213.nj/distributed-systems-basics-78292e1e437e>

[21] Birje, MN, Hanji, SS Nesnelerin interneti tabanlı dağıtılmış sağlık sistemleri: bir inceleme. *Veri, Bilgi ve Yönetim Dergisi* 2, 149–165 (2020). <https://doi.org/10.1007/s42488-020-00027-x>

[22] Shridhar Bhalekar, “Distributed Systems and Big Data Analytics in Predictive Healthcare: Transforming Modern Medicine”, *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol*, vol. 11, no. 2, pp. 2264–2283, Mar. 2025, doi: [10.32628/CSEIT25112713](https://doi.org/10.32628/CSEIT25112713).

[23] Sarina Aminizadeh, Arash Heidari, Mahshid Dehghan, Shiva Toumaj, Mahsa Rezaei, Nima Jafari Navimipour, Fabio Stroppa, Mehmet Unal,

[24] Abouelmehdi, K., Beni-Hessane, A. & Khaloufi, H. Big healthcare data: preserving security and privacy. *J Big Data* 5, 1 (2018). <https://doi.org/10.1186/s40537-017-0110-7>

[25] S. Firdous Turak, A. Jadhao and S. Makhijani, "AI Used in Healthcare and Data Privacy," 2024 2nd DMIHER International Conference on Artificial Intelligence in Healthcare, Education and Industry (IDICAIEI), Wardha, India, 2024, pp. 1-5, doi: 10.1109/IDICAIEI61867.2024.10842810

[26] <https://www.mevzuat.gov.tr/mevzuat?MevzuatNo=6698&MevzuatTur=1&MevzuatTertip=5>

[27] <https://link.springer.com/article/10.1007/S13042-022-01647-Y>

[28] <https://www.sciencedirect.com/science/article/pii/S0950705121000381>

[29] <https://tr.wikipedia.org/wiki/Bakteri>

[30] Article Source: [Deep learning approach to bacterial colony classification](#)  
Zieliński B, Plichta A, Misztal K, Spurek P, Brzychczy-Włoch M, et al. (2017) Deep learning

approach to bacterial colony classification. PLOS ONE 12(9): e0184554.

<https://doi.org/10.1371/journal.pone.0184554>

[31] <https://medium.com/@ilhnevval/convolutional-neural-network-cnn-evri%C5%9Fimli-sinir-a%C4%9Flar%C4%B1-fbc4a1988d2>

[32] [https://en.wikipedia.org/wiki/Convolutional\\_neural\\_network](https://en.wikipedia.org/wiki/Convolutional_neural_network)

[33] <https://medium.com/@tuncerergin/convolutional-neural-network-convnet-yada-cnn-nedir-nasil-calisir-97a0f5d34cad>

[34] R. C. Gonzalez, "Deep Convolutional Neural Networks [Lecture Notes]," in IEEE Signal Processing Magazine, vol. 35, no. 6, pp. 79-87, Nov. 2018, doi: 10.1109/MSP.2018.2842646.

[35] Khan, A., Sohail, A., Zahoor, U. *et al.* A survey of the recent architectures of deep convolutional neural networks. *Artif Intell Rev* 53, 5455–5516 (2020). <https://doi.org/10.1007/s10462-020-09825-6>

[36] <https://www.geeksforgeeks.org/deep-learning/residual-networks-resnet-deep-learning/>

[37] Weiss, K., Khoshgoftaar, T.M. & Wang, D. A survey of transfer learning. *J Big Data* 3, 9 (2016). <https://doi.org/10.1186/s40537-016-0043-6>

[38] <https://www.geeksforgeeks.org/machine-learning/ml-introduction-to-transfer-learning/>

[39] G. P. Reddy and Y. V. Pavan Kumar, "A Beginner's Guide to Federated Learning," *2023 Intelligent Methods, Systems, and Applications (IMSA)*, Giza, Egypt, 2023, . 557-562, doi: 10.1109/IMSA58542.2023.10217383.

[40] McMahan, B., Moore, E., Ramage, D., Hampson, S. & Arcas, B.A.y.. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, in *Proceedings of Machine Learning Research* 54:1273-1282 Available from <https://proceedings.mlr.press/v54/mcmahan17a.html>.

[41] <https://zkcplk.medium.com/federe-%C3%B6%C4%9Frenme-federated-learning-nedir-nas%C4%B1l-%C3%A7al%C4%B1%C5%9F%C4%B1r-fl04bdf3ea9b>

[42] <https://ai.nuhil.net/deep-learning/softmax>

[43] <https://www.geeksforgeeks.org/deep-learning/categorical-cross-entropy-in-multi-class-classification/>

- [44] <https://medium.com/@anuragkedia19/which-federated-learning-architecture-fits-the-best-95985ad40392>
- [46] <https://flower.ai/docs/baselines/fedprox.html>
- [47] <https://arxiv.org/abs/2111.04263>
- [48] <https://arxiv.org/abs/2003.00295>
- [49] <https://medium.com/mobillium/rest-api-restful-api-nedir-b45b32ab4a12>
- [50] <https://gokhana.medium.com/grpc-nedir-ve-nasıl-uygulanır-microservice-mimarisi-ile-grpc-9f1dc0847475>
- [51] <https://roltek.com.tr/blog/mqtt-nedir>
- [53] <https://medium.com/@EmirhanHasirci/google-colab-ile-model-e%C4%9Fitimini-h%C4%B1zland%C4%B1r%C4%B1n-a175d5d03731>
- [54] <https://canozdogan.medium.com/bulut-tabanlı-python-diledi%Finiz-verden-veri-analizi-yap%C4%B1n-google-colab-47fd5ded6ade>
- [56] *Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun*; Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778
- [57] Li Li, Yuxi Fan, Mike Tse, Kuo-Yi Lin A review of applications in federated learning. 2020
- [58] Chen Zhang, Yu Xie, Hang Bai, Bin Yu, Weihong Li, Yuan Gao, A survey on federated learning. 2020
- [60] [https://openmined.org/blog/federated-learning-types/?\\_x\\_tr\\_sl=en&\\_x\\_tr\\_tl=tr&\\_x\\_tr\\_hl=tr&\\_x\\_tr\\_pto=wa](https://openmined.org/blog/federated-learning-types/?_x_tr_sl=en&_x_tr_tl=tr&_x_tr_hl=tr&_x_tr_pto=wa)
- [61] <https://www.sciencedirect.com/science/article/pii/S0933365724000216>
- [62] <https://mustafaserdarkonca.medium.com/evri%C5%9Fimli-sinir-a%C4%9Flar%C4%B1-convolutional-neural-networks-cnn-74b3d4a567f9>
- [63] [https://en.wikipedia.org/wiki/Federated\\_learning](https://en.wikipedia.org/wiki/Federated_learning)
- [64] [https://www.researchgate.net/figure/Diagram-of-Federated-Learning-training-process\\_fig1\\_373715360](https://www.researchgate.net/figure/Diagram-of-Federated-Learning-training-process_fig1_373715360)
- [65] <https://www.blog.trainindata.com/one-hot-encoding-categorical-variables/>
- [66] Gómez Pinilla, E. F. (2013). Diseño de estrategia competitiva como eje de crecimiento de Aidoc Solution Sas. (2013).

- [67] <https://dergipark.org.tr/en/pub/sdusyd/article/842035>
- [68] Shin, S. Y. (2019). Current status and future direction of digital health in Korea. *The Korean Journal of Physiology & Pharmacology*, 23(5), 311–315.
- [69] Chen, M., Mathews, R., Ouyang, T., & Beaufays, F. (2019). Federated learning of out-of-vocabulary words. *arXiv preprint arXiv:1903.10635*.\*
- [70] Leroy, A., Coucke, T., Lavril, T., Gisselbrecht, T., & Dureau, J. (2019). Federated learning for keyword spotting. *arXiv preprint arXiv:1810.05512*.\*
- [71] Hard, A., Rao, K., Mathews, R., Ramaswamy, S., Beaufays, F., Augenstein, S., & Ramage, D. (2018). Federated learning for mobile keyboard prediction. *arXiv preprint arXiv:1811.03604*.\*
- [72] Yang, T., Andrew, G., Eichner, H., Sun, H., Li, W., Kong, N., Ramage, D., & Beaufays, F. (2018). Applied federated learning: Improving Google keyboard query suggestions. *arXiv preprint arXiv:1812.02903*.\*
- [73] Ramaswamy, S., Mathews, R., Rao, K., & Beaufays, F. (2019). Federated Learning for Emoji Prediction in a Mobile Keyboard.
- [74] Sozinov, K., Vlassov, V., & Girdzijauskas, S. (2018). Human Activity Recognition Using Federated Learning. In *Proceedings of the 2018 IEEE International Conference on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCLOUD/SocialCom/SustainCom)* (pp. 1103–1111). IEEE.
- [75] Wang, X., Han, Y., Wang, C., Zhao, Q., Chen, X., & Chen, M. (2019). In-Edge AI: Intelligentizing Mobile Edge Computing, Caching and Communication by Federated Learning. *IEEE Network*, 33(3), 156–165.
- [76] Qian, Y., Hu, L., Chen, J., Guan, X., Hassan, M. M., & Alelaiwi, A. (2019). Privacy-aware service placement for mobile edge computing via federated learning. *Information Sciences*, 505, 562–570.
- [77] Aïvodji, U. M., Gambs, S. & Martin, A. (2019). IOTFLA: A secured and privacy-preserving smart home architecture implementing federated learning. In *2019 IEEE Security and Privacy Workshops (SPW)* (pp. 175-180). IEEE.
- [78] Hu, B., Gao, Y., Liu, L. & Ma, H. (2018). Federated Region-Learning: An Edge Computing Based Framework for Urban Environment Sensing. In *Proceedings of the IEEE Global Communications Conference (GLOBECOM 2018)*, Abu Dhabi, United Arab Emirates, 9–13 December 2018, pp. 1–7.

- [79] Han, X., Yu, H. & Gu, H. (2019). Visual Inspection with Federated Learning. In Image Analysis and Recognition: 16th International Conference, ICIAR 2019 (pp. 52–64). Springer International Publishing.
- [80] Liu, Y., Wu, Z., Ge, S., Fan, Y., & Zou, Y. (2020). Federated Multimodal Representation Learning for Vision–Language Grounding. In Proceedings of the AAAI Conference on Artificial Intelligence (AAAI-20).
- [81] Mowla, N. I., Tran, N. H., Doh, I. & Chae, K. (2020). Federated Learning-Based Cognitive Detection of Jamming Attack in Flying Ad-Hoc Network. IEEE Access, 8, 4338–4350.
- [82] Liu, Y., Wu, Y., Ge, S., Fan, Z., & Zou, Y. (2020). Federated Learning for Vision-and-Language Grounding. arXiv preprint arXiv:2008.06221.
- [83] Yang, Q., Zhang, J., Ye, J., Li, J., & Xu, C. (2019). Federated Machine Learning: Concept and Applications. ACM Transactions on Intelligent Systems and Technology (TIST), 10(2), 1–19.
- [84] Wang, J., Tong, Y., & Shi, J. (2020). Federated Latent Dirichlet Allocation for Industrial-Scale Text Mining. IEEE Transactions on Industrial Informatics.
- [85] Liu, Y., James, J. Q., Kang, J., Niyato, D., & Zhang, S. (2020). Privacy-Preserving Traffic Flow Prediction Using Federated Learning. IEEE Transactions on Intelligent Transportation Systems.
- [86] Liu, J., Dligach, D., & Miller, T. (2019). Federated Learning for Clinical Concept Extraction. Proceedings of the ACL Workshop on Clinical NLP.
- [87] Kim, M., Sun, J., Yu, H., & Jiang, X. (2017). Federated tensor factorization for computational phenotyping. KDD 2017 – Knowledge Discovery and Data Mining.

[88] Lee, J., Sun, J., Wang, F., Wang, S., Jun, C. H., & Jiang, X. (2018). Privacy-Preserving Patient Similarity Learning in a Federated Environment: Development and Analysis. *JMIR Medical Informatics*, 6(2), e20.

[89] Cohen, I. G., Mello, M. M., & Gostin, L. O. (2019). Big data, big problems: Protecting patient privacy while harnessing the power of health data. *Journal of the American Medical Association (JAMA)*, 322(6), 507–508.

[90] Silva, S., Gutman, B., Romero, E., Thompson, P. M., Altmann, A., & Lorenzi, M. (2019).

Federated learning in distributed medical databases: Meta-analysis of large-scale subcortical brain data.

*NeuroImage*, 194, 75–89.

[91] Gao, D., Liu, Y., Wang, X., Lin, H., Ma, X., & Yang, Q. (2019).

Hierarchical heterogeneous federated learning for EEG classification.

*IEEE International Conference on Big Data (Big Data 2019)*.

[92] Pfohl, S. R., Dai, A. M., & Heller, K. A. (2019).

Federated learning with differential privacy for electronic medical records.

arXiv preprint arXiv:1911.05861

[93] Huang, L., Shea, A. L., Qian, H., Masurkar, A., Deng, H., & Liu, D. (2019).

Patient clustering improves efficiency of federated machine learning to predict mortality and hospital stay time using distributed electronic medical records.

*Journal of Biomedical Informatics*, 99, 103291

[94] Brisimi, T. S., Chen, R., Mela, T., Olshevsky, A., Paschalidis, I. C., & Shi, W. (2018).

Federated learning of predictive models from federated electronic health records.

*International Journal of Medical Informatics*, 112, 59–67

[95] Huang, L., Yin, Y., Fu, Z., Zhang, S., Deng, H. & Liu, D. (2020). LoAdaBoost: Loss-based AdaBoost federated machine learning with reduced computational complexity on IID and non-IID intensive care data. *PLoS ONE*, 15(4): e0230706.

[96] Dayan, I., Roth, H. R., Zhong, A., Harouni, A., Gentili, A., Abidin, A. Z., Liu, A., Costa, A. B., Wood, B. J., Tsai, C.-S., Gilbert, F. J., Li, Q., Dreyer, K. J. & Flores, M. G. (2021).

Federated learning for predicting clinical outcomes in patients with COVID-19. *Nature Medicine*, 27(10), 1735–1743.

[97] Ouyang, L., Yuan, Y., Cao, Y. & Wang, F.-Y. (2021). A novel framework of collaborative early warning for COVID-19 based on blockchain and smart contracts. *Information Sciences*, 570, 124–143.

[98] Ma, X., Zhang, Y., Liu, Q., Yang, Z., Li, H., Wang, J., Hua, W. & Li, Y. (2022). An Assisted Diagnosis Model for Cancer Patients Based on Federated Learning.

EKLER

[1] <https://github.com/iakaradeniz/FederatedLearningSimulation>

## ÖZGEÇMİŞ

Muhammed Yusuf Öngel, 26 Kasım 2004 yılında Yozgat'ta doğdu. İlk, orta ve lise eğitimini Sincan'da tamamladı. 2022 yılında Erkan Volkan Yiğit Anadolu İmam Hatip Lisesi'nden mezun oldu. Aynı yılda da Sakarya Üniversitesi Bilgisayar Mühendisliği bölümünü kazandı. Yazılım ve donanım stajını 2025'in yaz ayında Ankara Teknopark'taki VHS Elektronik firmasında yaptı. Şu anda lisans eğitiminin 4. sınıfına devam etmektedir.

İsmail Alper Karadeniz, 31 Mart 2003'de Konya'da doğdu. İlk, orta ve lise eğitimini Konya'da tamamladı. 2021 yılında Konya Şehir Koleji'nden mezun oldu. 2022 yılında Sakarya Üniversitesi Bilgisayar Mühendisliği bölümünü kazandı. Şu anda lisans eğitiminin 4. sınıfına devam etmektedir.

## BSM 401 BİLGİSAYAR MÜHENDİSLİĞİ TASARIMI DEĞERLENDİRME VE SÖZLÜ SINAV TUTANAĞI

KONU: Sağlık Sektöründe Federatif Öğrenme Tabanlı Yapay Zeka

ÖĞRENCİLER (Öğrenci No/Ad/Soyad): b221210100 Muhammed Yusuf Öngel, b221210065 İsmail

Alper Karadeniz

| Değerlendirme Konusu                                                                                                                           | İstenenler | Not Aralığı | Not |
|------------------------------------------------------------------------------------------------------------------------------------------------|------------|-------------|-----|
| <b>Yazılı Çalışma</b>                                                                                                                          |            |             |     |
| Çalışma klavuza uygun olarak hazırlanmış mı?                                                                                                   | x          | 0-5         |     |
| <b>Teknik Yönden</b>                                                                                                                           |            |             |     |
| Problemin tanımı yapılmış mı?                                                                                                                  | x          | 0-5         |     |
| Geliştirilecek yazılımın/donanımın mimarisini içeren blok şeması (yazılımlar için veri akış şeması (dfd) da olabilir) çizilerek açıklanmış mı? | x          | 0-10        |     |
| Blok şemadaki birimler arasındaki bilgi akışına ait model/gösterim var mı?                                                                     |            |             |     |
| Yazılımın gereksinim listesi oluşturulmuş mu?                                                                                                  | x          |             |     |
| Kullanılan/kullanılması düşünülen araçlar/teknolojiler anlatılmış mı?                                                                          | x          | 0-10        |     |
| Donanımların programlanması/konfigürasyonu için yazılım gereksinimleri belirtilmiş mi?                                                         |            |             |     |
| UML ile modelleme yapılmış mı?                                                                                                                 |            |             |     |
| Veritabanları kullanılmış ise kavramsal model çıkarılmış mı? (Varlık ilişki modeli, noSQL kavramsal modelleri v.b.)                            |            |             |     |
| Projeye yönelik iş-zaman çizelgesi çıkarılarak maliyet analizi yapılmış mı?                                                                    | x          | 0-10        |     |
| Donanım bileşenlerinin maliyet analizi (prototip-adetli seri üretim vb.) çıkarılmış mı?                                                        |            |             |     |
| Donanım için gerekli enerji analizi (minimum-uyku-aktif-maksimum) yapılmış mı?                                                                 |            |             |     |
| Grup çalışmalarında grup üyelerinin görev tanımları verilmiş mi (iş-zaman çizelgesinde belirtilebilir)?                                        | x          | 0-5         |     |
| Sürüm denetim sistemi (Version Control System; Git, Subversion v.s.) kullanılmış mı?                                                           |            |             |     |
| Sistemin genel testi için uygulanan metotlar ve iyileştirme süreçlerinin dökümü verilmiş mi?                                                   | x          | 0-5         |     |
| Yazılımın sızma testi yapılmış mı?                                                                                                             |            |             |     |
| Performans testi yapılmış mı?                                                                                                                  |            |             |     |
| Tasarımın uygulamasında ortaya çıkan uyumsuzluklar ve aksaklıklar belirtilerek çözüm yöntemleri tartışılmış mı?                                | x          | 0-5         |     |
| Yapılan işlerin zorluk derecesi?                                                                                                               | x          | 0-15        |     |
| <b>Sözlü Sınav</b>                                                                                                                             |            |             |     |
| Yapılan sunum başarılı mı?                                                                                                                     |            |             |     |
| Soruları yanıtlama yetkinliği?                                                                                                                 |            |             |     |
| <b>Devam Durumu</b>                                                                                                                            |            |             |     |
| Öğrenci dönem içerisindeki raporlarını düzenli olarak hazırladı mı?                                                                            | x          | 0-15        |     |
| <b>Diğer Maddeler</b>                                                                                                                          |            |             |     |
| Veri Hazırlanması                                                                                                                              |            |             |     |

|               |   |      |  |
|---------------|---|------|--|
| Yöntemler     | x | 0-15 |  |
|               |   |      |  |
| <b>Toplam</b> |   |      |  |

DANIŞMAN : DR. ÖĞR. ÜYESİ İSMAİL ÖZTEL  
DANIŞMAN İMZASI: